

DEG: Efficient Hybrid Vector Search Using the Dynamic Edge Navigation Graph

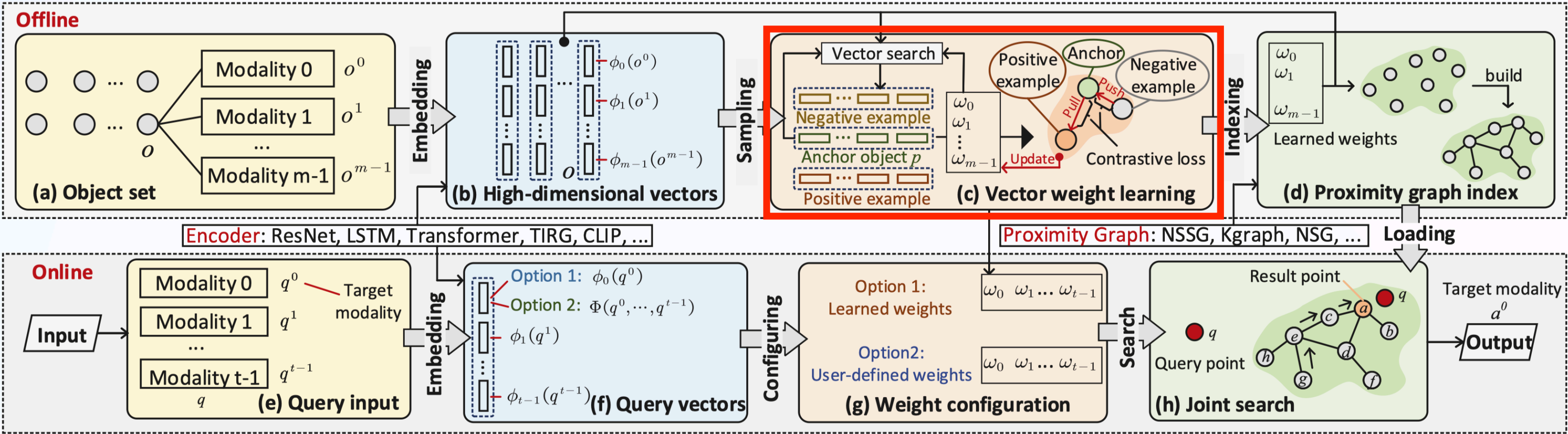
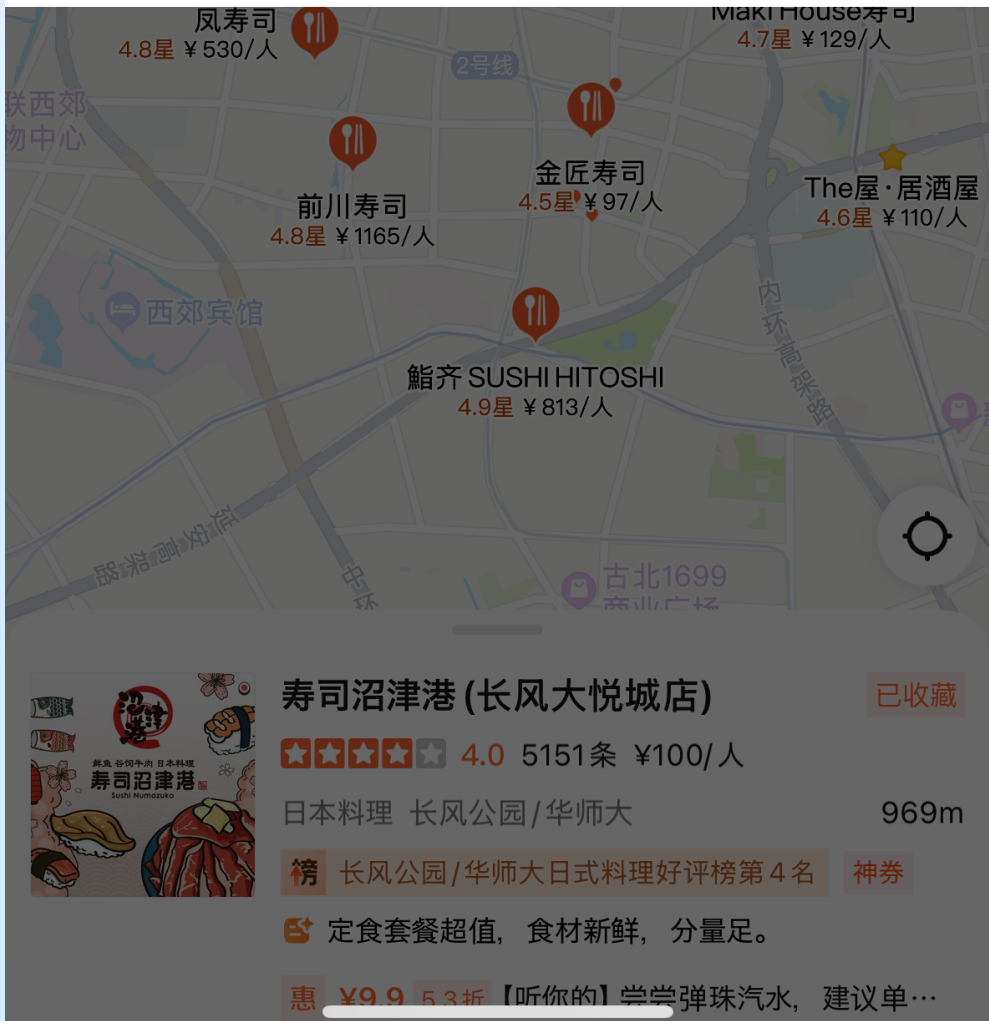
ZIQI YIN, JIANYANG GAO, PASQUALE BALSEBRE, GAO CONG, CHENG LONG
Nanyang Technological University, Singapore

Proc. ACM Manag. Data, Vol. 3, No. 1 (SIGMOD), Article 29. Publication date: February 2025.

Motivation

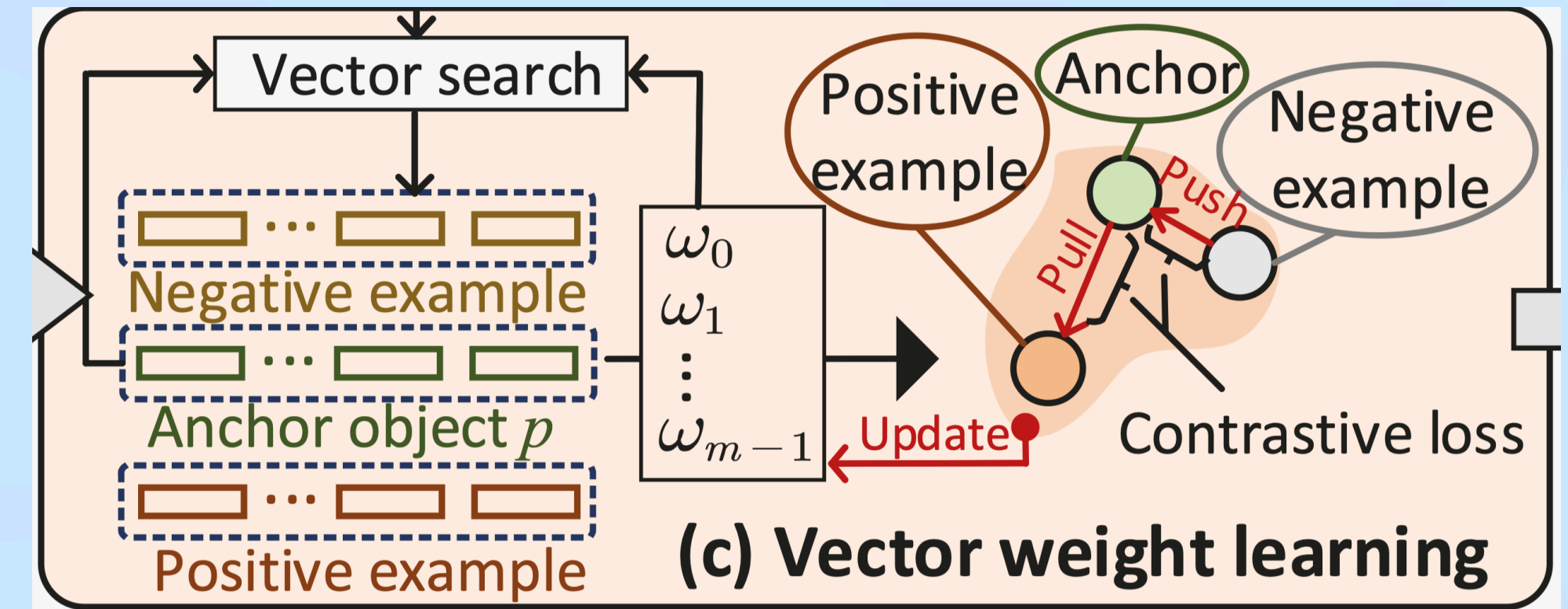
Why we need dynamic α in Multi-Vector Top-K Search

$$Dist(q, o) = q.\alpha \times \delta_e(q, o) + (1 - q.\alpha) \times \delta_s(q, o),$$



Motivation

MUST: Vector Weight Learning



Concatenated vector for both data and query

$$\hat{p} = [\omega_0 \phi_0(p^0), \dots, \omega_{m-1} \phi_{m-1}(p^{m-1})]$$

- p^i is the data/query of i -th modality.
- ϕ_i is the encoder (embedding function) of i -th modality.
- w_i is the **learnable** weights of i -th modality.

Background

Multi-Vector Top-K Search

Hybrid Vector Query (HVQ): Given a dataset D , a hybrid vector query $q = \langle e, s, \alpha, k \rangle$ consists of two feature vectors $q.e \in E^d$ and $q.s \in E^m$, a hyperparameter $q.\alpha \in [0, 1]$, and the number of objects to be returned $q.k$. HVQ aims to retrieve k objects with the minimum distance $Dist(q, o)$:

$$Dist(q, o) = q.\alpha \times \delta_e(q, o) + (1 - q.\alpha) \times \delta_s(q, o), \quad (1)$$

- **Approach 1. Vector Fusion.** Build index on (weighted) sum of vector.
- **Approach 2. Vector Merge.** Build independent index on vectors, then merge and re-rank all objects in query phase.

Both approaches can only built on fixed α value on **indexing**.

Background

NRA (Non-Random Access) Algorithm

L_1	L_2
$(R, 1)$	$(\cdot, \frac{1}{3})$
$(\cdot, \frac{1}{3})$	$\dots$
$\dots$	$(\cdot, \frac{1}{3})$
$(\cdot, \frac{1}{3})$	$(R, 0)$

Example 8.3: Consider the following scenario, where the aggregation function is the average, and where $k = 1$ (so that we are interested only in the top object). There are only two sorted lists L_1 and L_2 (see Figure 4), and the grade of every object in both L_1 and L_2 is $1/3$, except that object R has grade 1 in L_1 and grade 0 in L_2 . After two sorted accesses to L_1 and one sorted access to L_2 , there is enough information to know that object R is the top object (its average grade is at least $1/2$, and every other object has average grade at most $1/3$). If we wished to find the grade of object R , we would need to do sorted access to all of L_2 . $\square$

Background

NRA (Non-Random Access) Algorithm: Batch Optimization

Algorithm 2: Iterative merging

```
1  $k' \leftarrow k$ ;  
2 while  $k' < \text{threshold}$  do  
   // run top- $k'$  processing for each  $q.v_i$  on  $\mathcal{D}_i$   
3   foreach  $i$  do  
4      $\mathcal{R}_i \leftarrow \text{VectorQuery}(q.v_i, \mathcal{D}_i, k')$ ;  
5   if  $k$  results are fully determined with NRA [19] on all  $\mathcal{R}_i$   
   then  
6     return top- $k$  results;  
7   else  
8      $k' \leftarrow k' \times 2$ ;  
9 return top- $k$  results from  $\cup_i \mathcal{R}_i$ ;
```

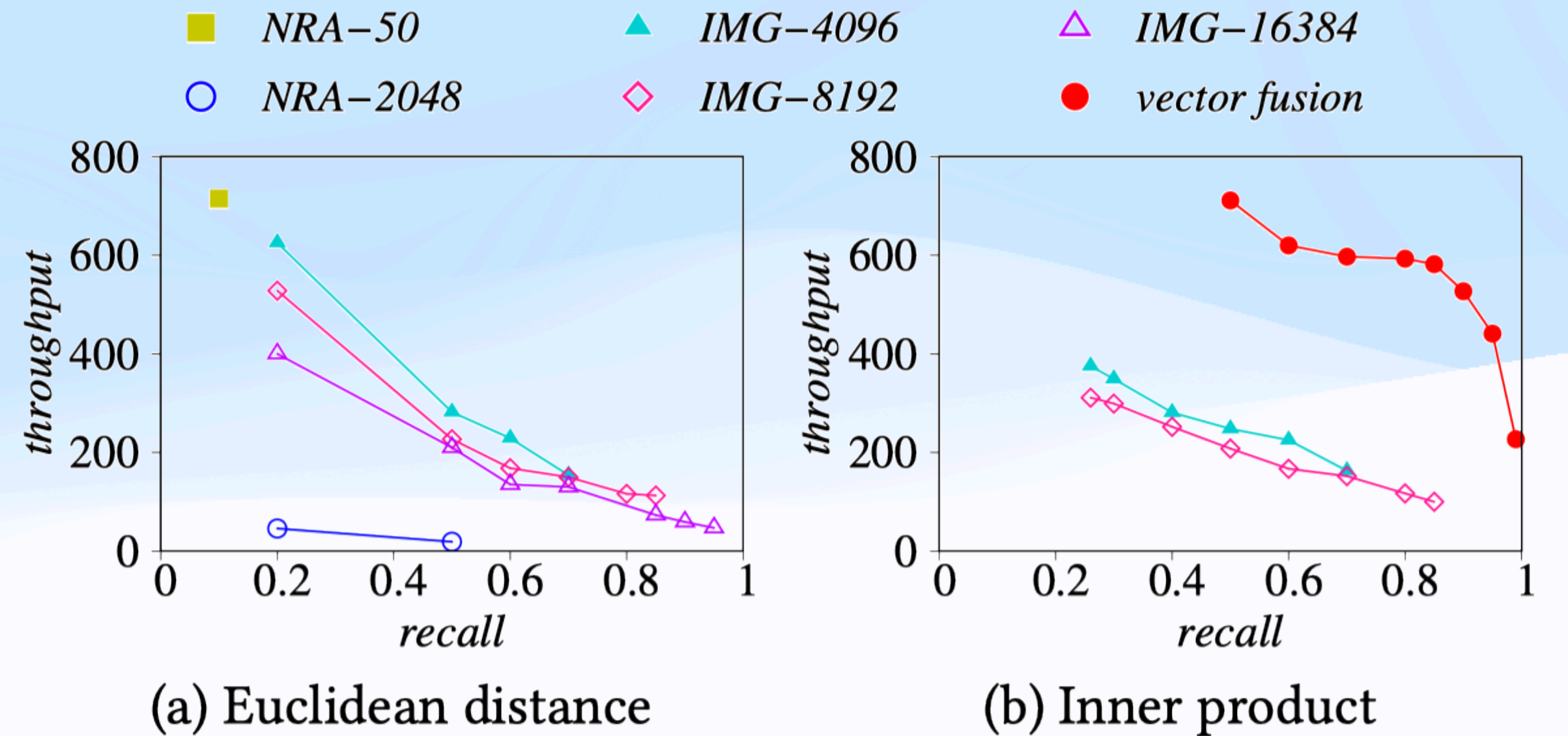
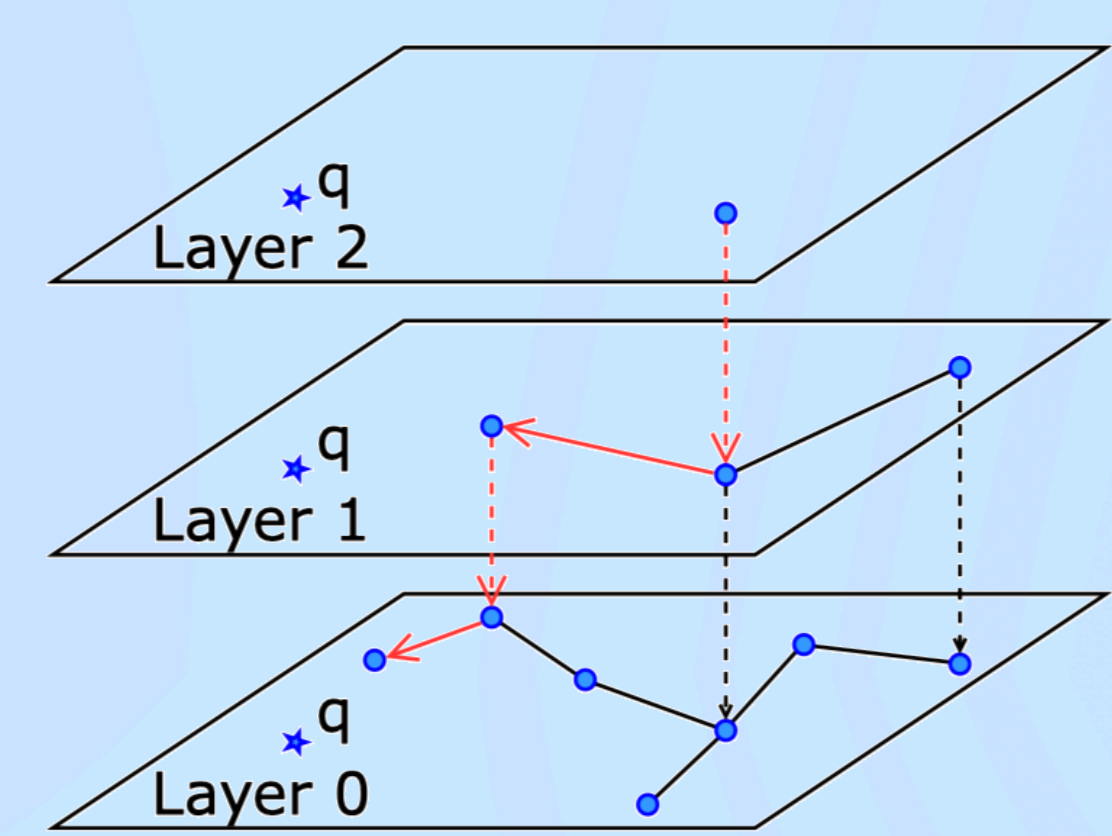
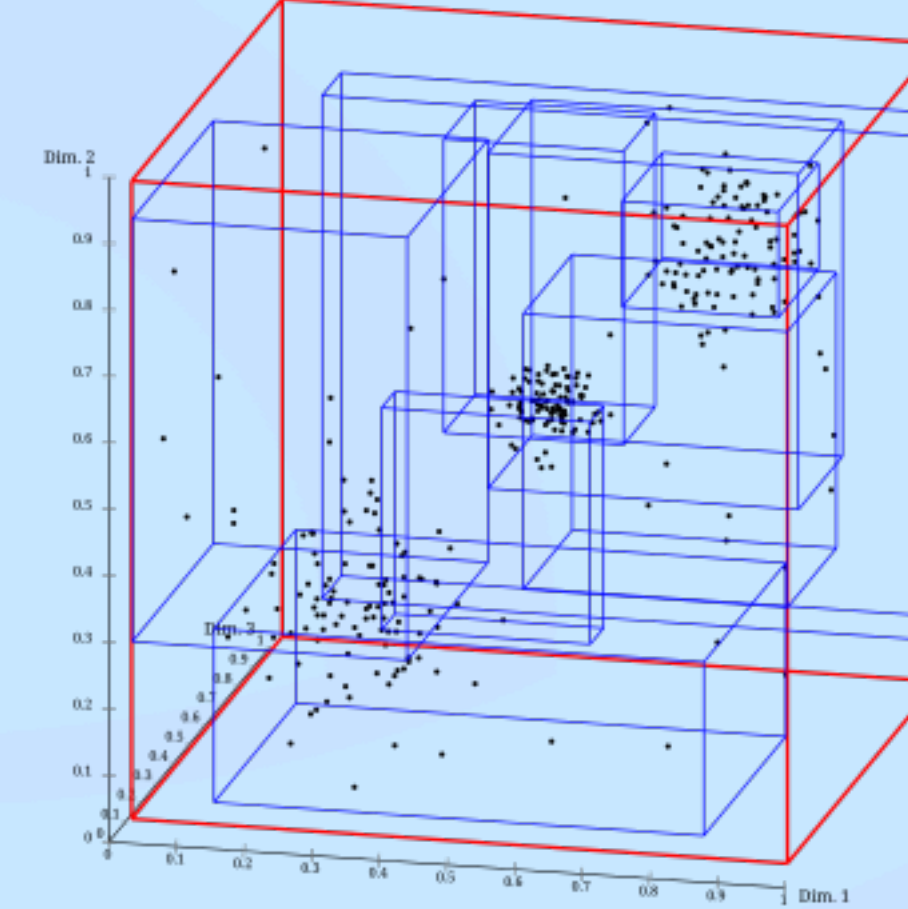


Figure 16: Multi-vector processing in Milvus

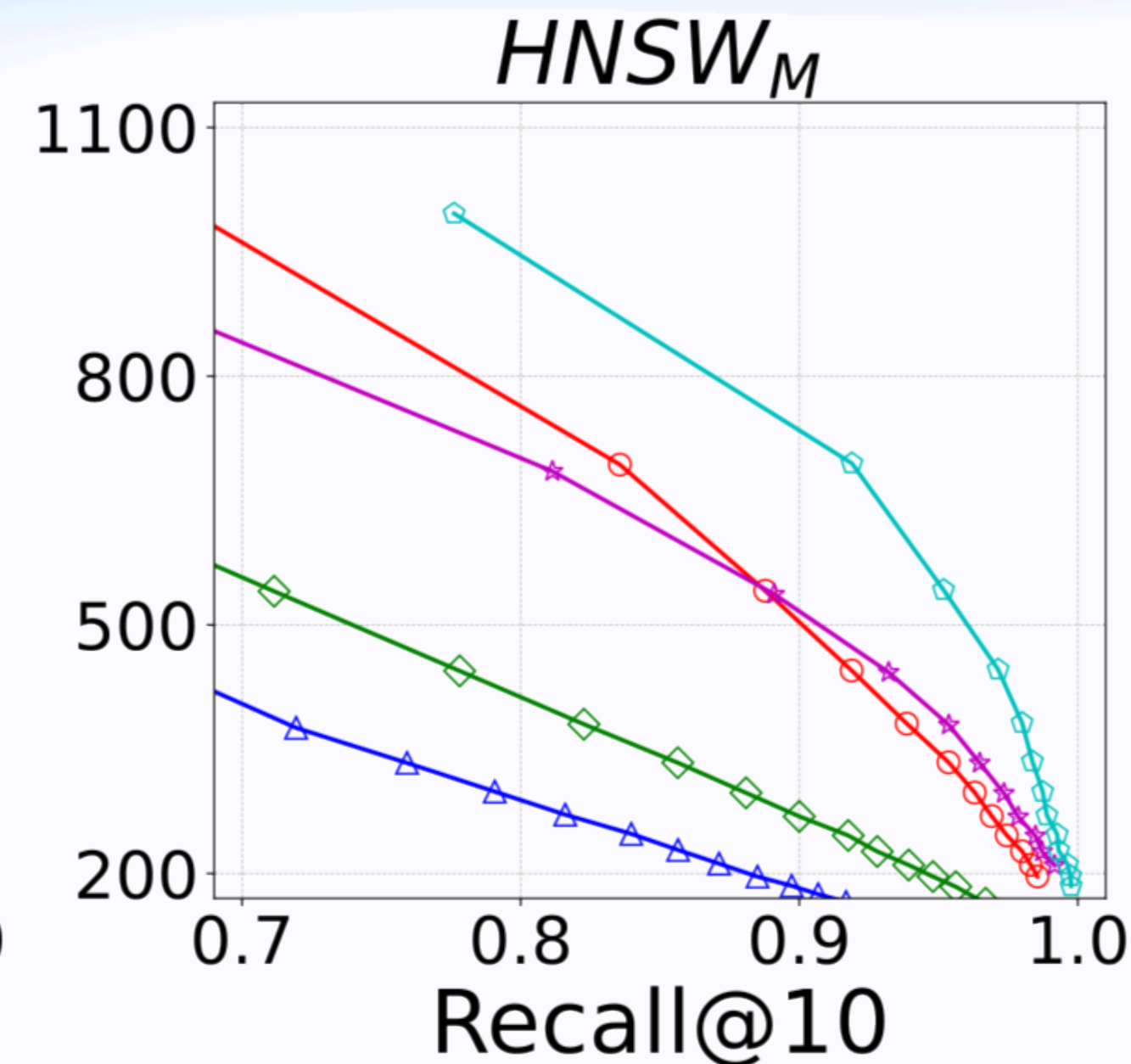
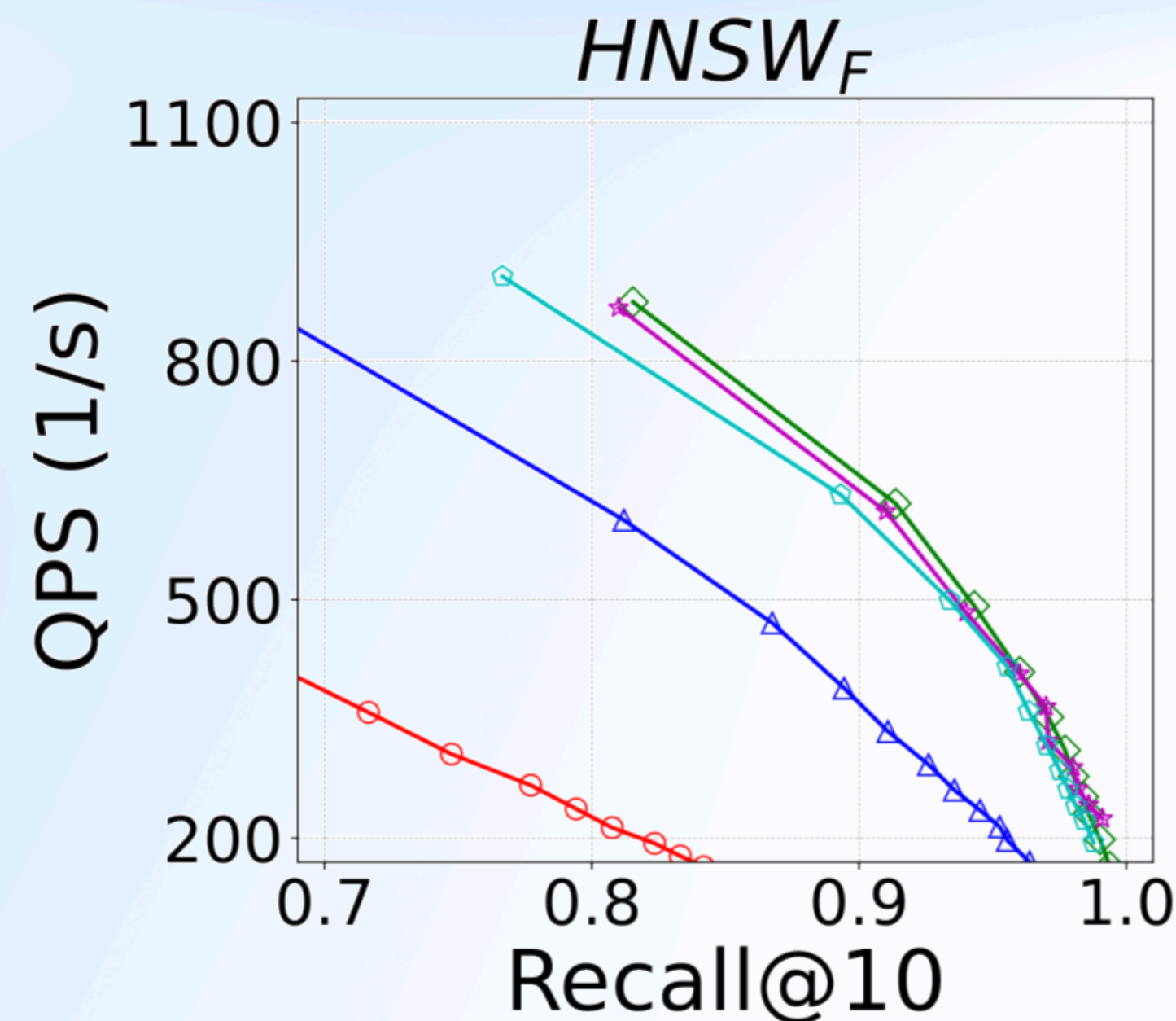
Background

Baseline Evaluation

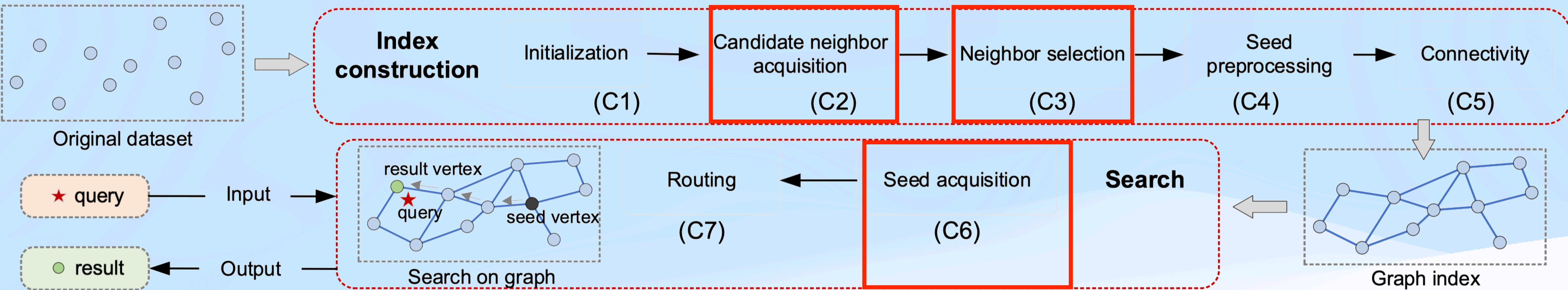


- **HNSW-F**: HNSF Indexing + Vector Fusion Method (**best for $\alpha = 0.5$**)
- **HNSW-M**: HNSF/R-Tree Indexing + Merging Method (Not NRA) (**best for $\alpha = 0/1$**)

$\alpha=0.1$ $\alpha=0.3$ $\alpha=0.5$ $\alpha=0.7$ $\alpha=0.9$



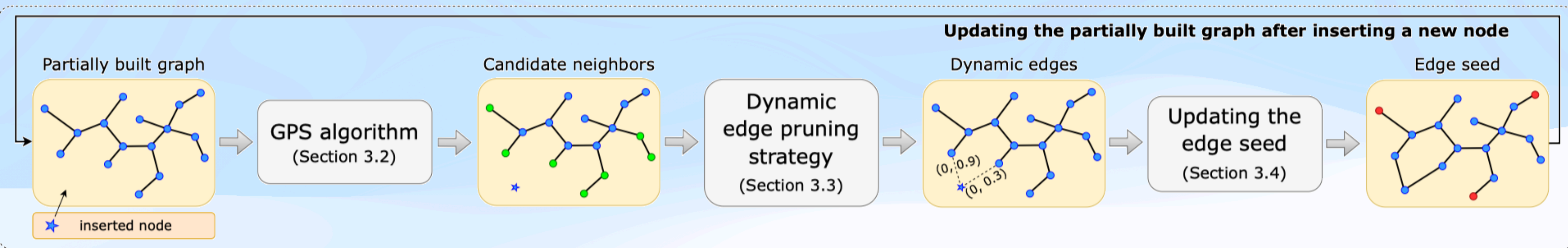
Challenges



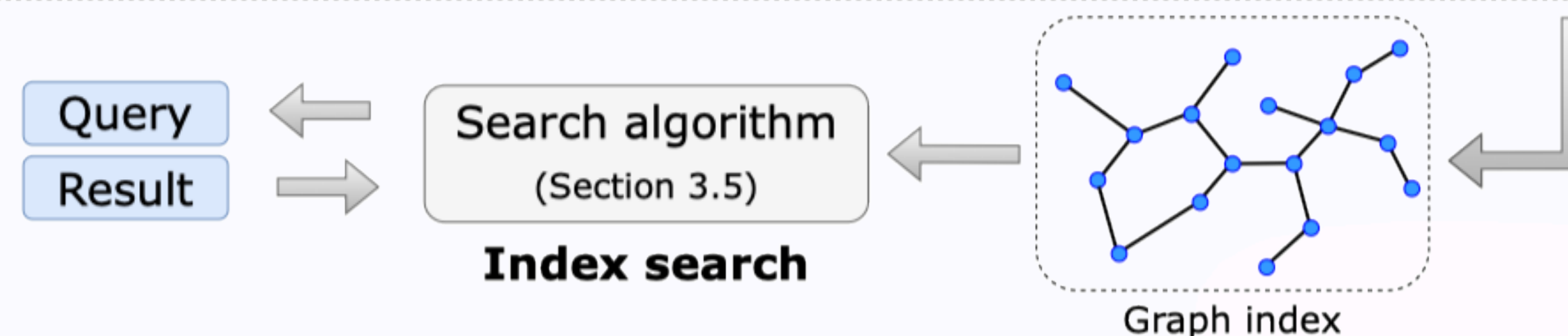
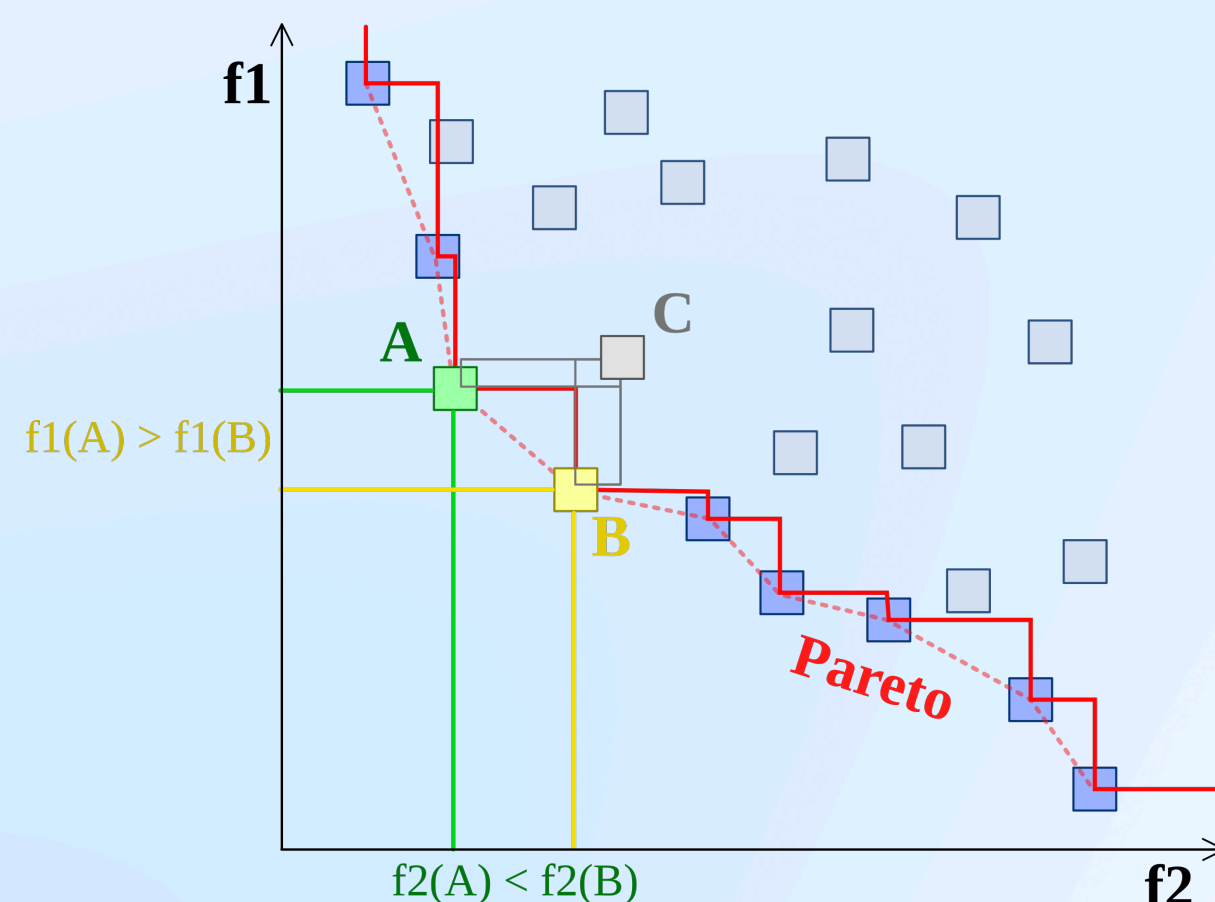
1. How to compute the **candidate neighbor** set for each node. (C2)
2. How to determine **edges** from the candidate set. (C3)
3. How to select the **seed** for the graph index in the HVQ problem. (C6)

Framework

Enhancements in Graph-Based Vector ANNS Workflow

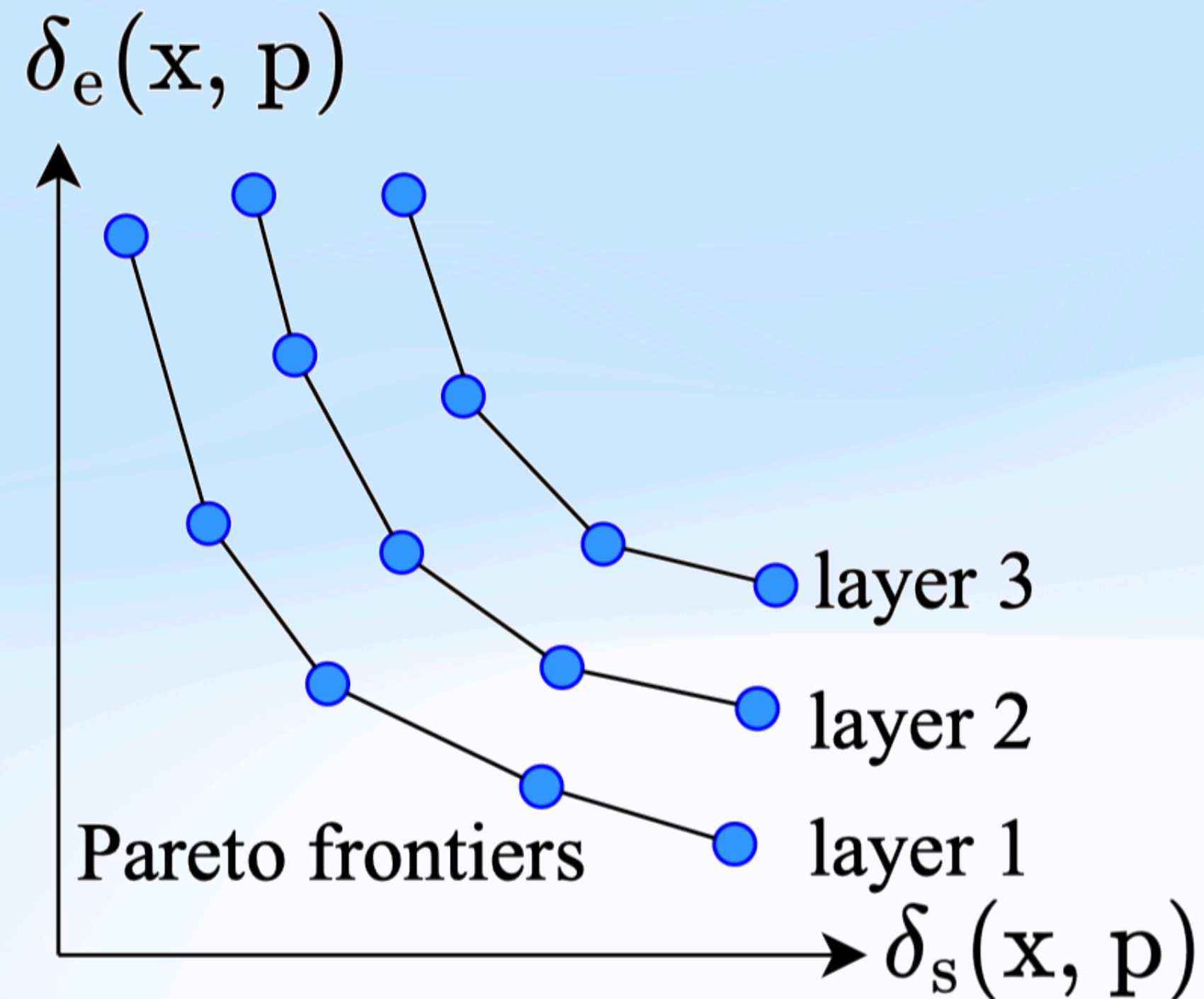


Index Construction: Inserting each node one by one



Stage 1. Candidate Neighbor Selection

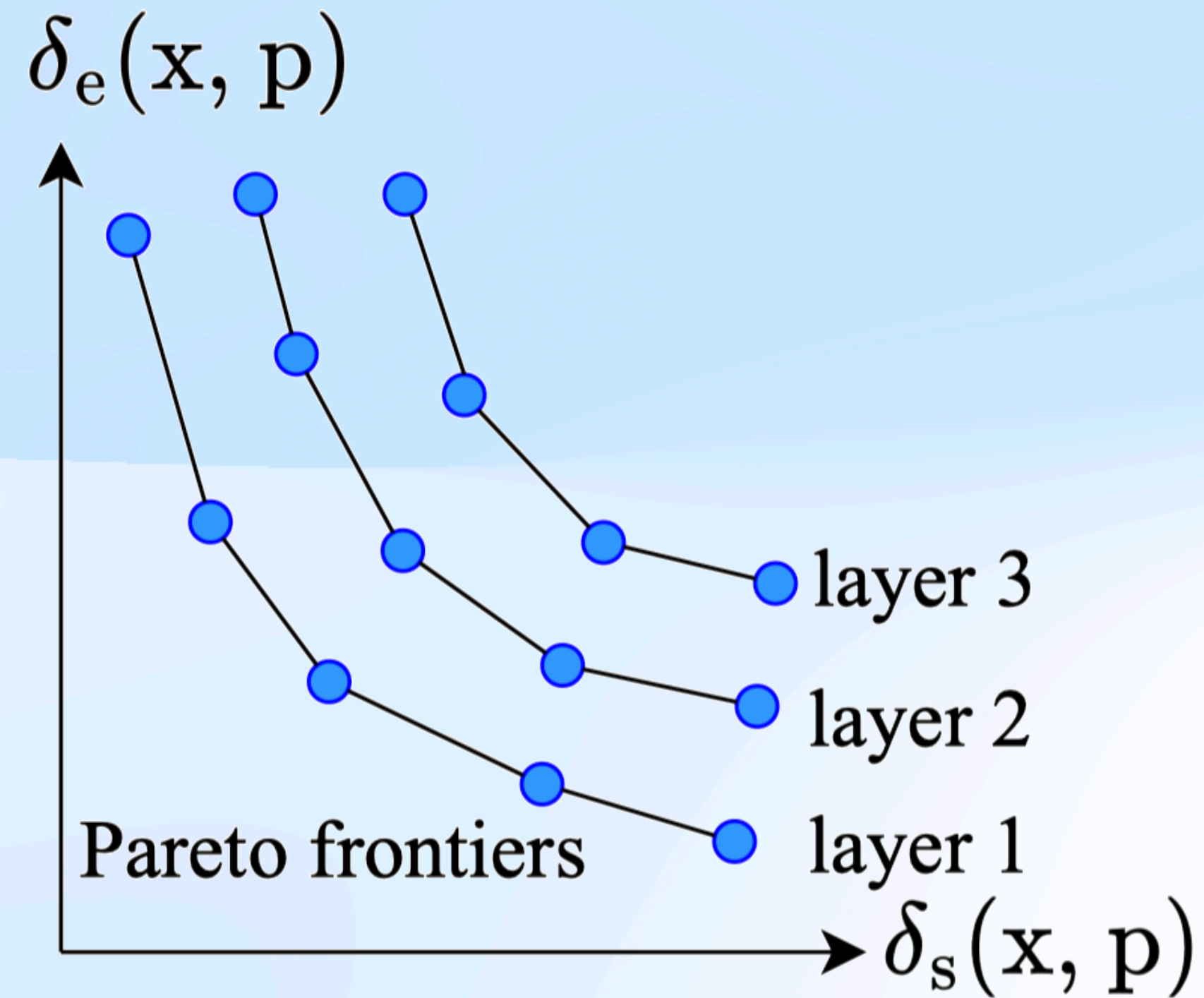
GPS (Greedy Pareto Frontier Search) Algorithm



Pareto Frontier [45]. Consider a dataset D and a multi-objective optimization problem described by $f(x): x \rightarrow \mathbb{R}^c$, where $x \in D$. Each function $f_i(x) \rightarrow \mathbb{R}$ represents the objective function of the i -th task to be minimized, with $i \in C$ and $C = \{1, 2, \dots, c\}$. For any $x, y \in D$, x dominates y if and only if (1) $\forall i \in C, f_i(x) \leq f_i(y)$ and (2) $\exists j \in C, f_j(x) < f_j(y)$. An object $x \in D$ is considered Pareto optimal if no other object in D dominates x . The Pareto frontier, also called the Pareto set, comprises all Pareto optimal objects in D .

Stage 1. Candidate Neighbor Selection

GPS (Greedy Pareto Frontier Search) Algorithm



Algorithm 1: GPS($G, q, ep, ef_{construction}$)

Input: A partially built graph G , a query q , start node set ep , candidate pool size $ef_{construction}$

Output: Multi-layer Pareto frontiers $\{PF^1(q), \dots, PF^l(q)\}$ for q , where $\sum_{i=1}^l |PF^i(q)| \leq ef_{construction}$

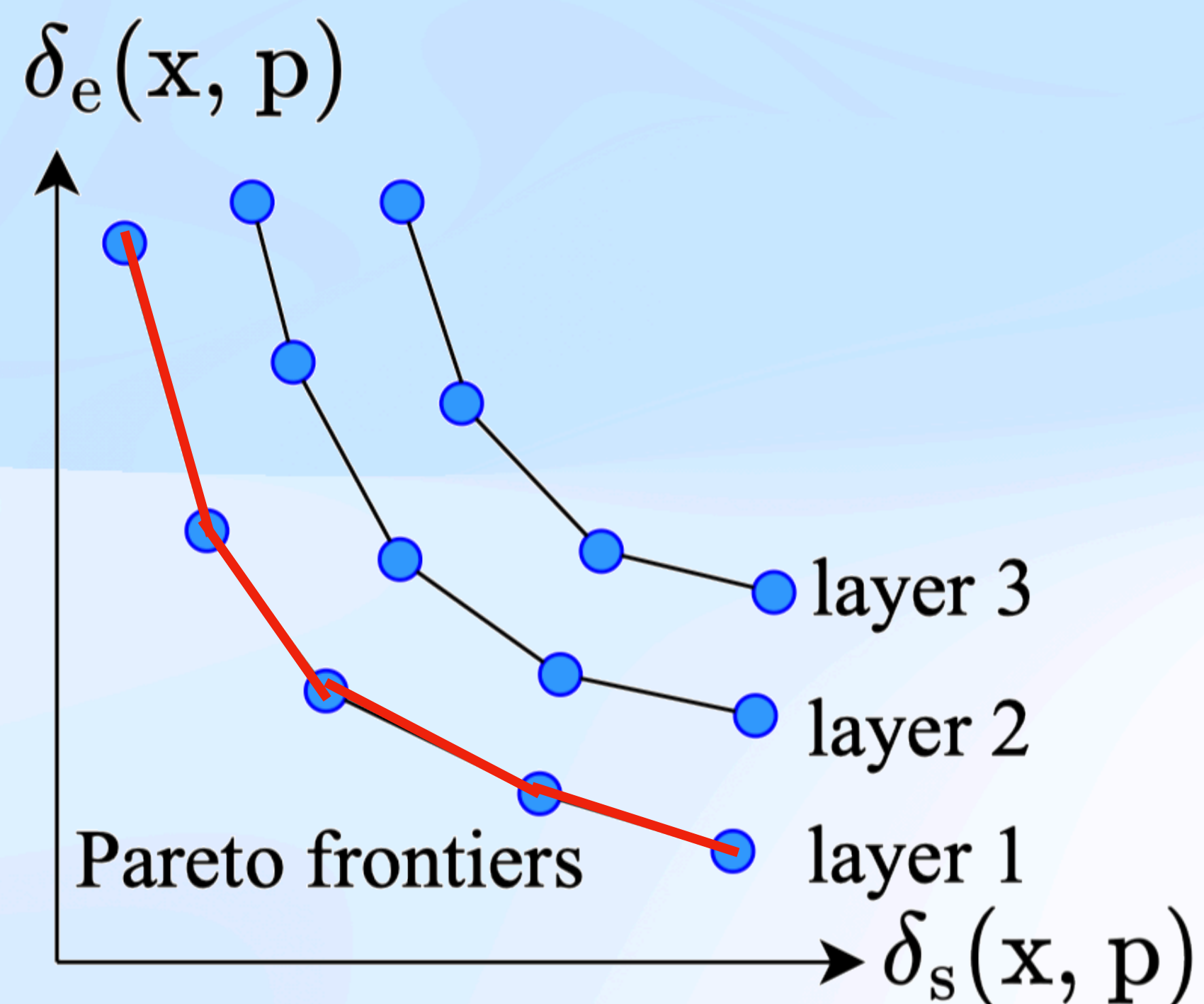
```

1 Procedure GPS( $G, q, ep, ef_{construction}$ ):
2    $Res \leftarrow \emptyset$ ;                                     // set of the pareto frontiers
3    $Vis \leftarrow \emptyset$ ;                               // set of the visited nodes
4    $Flag \leftarrow \emptyset$ ;                               // set of the explored nodes
5   foreach  $v \in ep$  do
6      $Res.add(v)$ ;
7      $Vis.add(v)$ ;
8    $Res \leftarrow FindPF(Res, ef_{construction});$ 
9   while  $|Res| < ef_{construction}$  do
10     $NNS \leftarrow \emptyset$ ;                               // set of unexplored new nodes
11    foreach  $S_k \in Res$  do
12       $NNS \leftarrow \{v \in S_k \mid v \notin Flag\}$ ;
13      if  $NNS \neq \emptyset$  then
14        break;
15    if  $NNS = \emptyset$  then
16      break;
17    foreach  $u \in NNS$  do
18       $Flag.add(u)$ ;                                       // mark  $u$  as old node
19      foreach  $v \in neighbour(u)$  do
20        if  $v \notin vis$  then
21           $Res.add(v)$ ;
22           $Vis.add(v)$ ;
23    // update the pareto frontiers using existing algorithm
24     $Res \leftarrow FindPF(Res, ef_{construction});$ 
25  return  $Res$ ;

```

Stage 1. Candidate Neighbor Selection

GPS (Greedy Pareto Frontier Search) Algorithm



Algorithm 4: Finding Pareto Frontiers

Input: A candidate pool CS for node p , candidate pool size $ef_{construction}$

Output: l layer Pareto frontiers $\{PF^1(p), \dots, PF^l(p)\}$ for node p , where $\sum_{i=1}^l |PF^i(p)| \leq ef_{construction}$

```

1 Procedure FindPF( $CS, ef_{construction}$ ):
2   Sort  $x \in CS$  in ascending order of  $\delta_s(x, p)$  to  $p$ ;
3    $Res \leftarrow \emptyset$ ;                                     // result set
4   while  $|Res| < ef_{construction}$  do
5      $PF \leftarrow \emptyset$ ;                               // current pareto frontier
6      $Remain \leftarrow \emptyset$ ;                          // remaining candidates
7      $PrevEmbDist \leftarrow \infty$ ;                       // smallest  $\delta_e(x, p)$ ,  $x \in PF$ 
8     foreach  $x \in CS$  do
9       if  $\delta_e(x, p) < PrevEmbDist$  then
10         $PF.add(x)$ ;
11         $PrevEmbDist \leftarrow \delta_e(x, p)$ ;
12       else
13         $Remain.add(x)$ ;
14       if  $|PF| + |Res| < ef_{construction}$  then
15         $Res.add(PF)$ ;
16         $CS \leftarrow Remain$ ;
17       else
18        break;
19   return  $Res$ ;

```

Take an example.

(1,5), (4, 3), (2, 2), (3, 4), (5, 1)

Sort based on $A[0]$ we have:

(1, 5), (2, 2), (3, 4), (4, 3), (5, 1)

Then look for the descending $A[1]$:

(1, 5), (2, 2), (5, 1)

Stage 2. Neighborhood Selection

Dynamic Edge Pruning Strategy: RNG in 2 Vectors

$$\alpha \times \delta_e(x, z) + (1 - \alpha) \times \delta_s(x, z) < \quad (2)$$

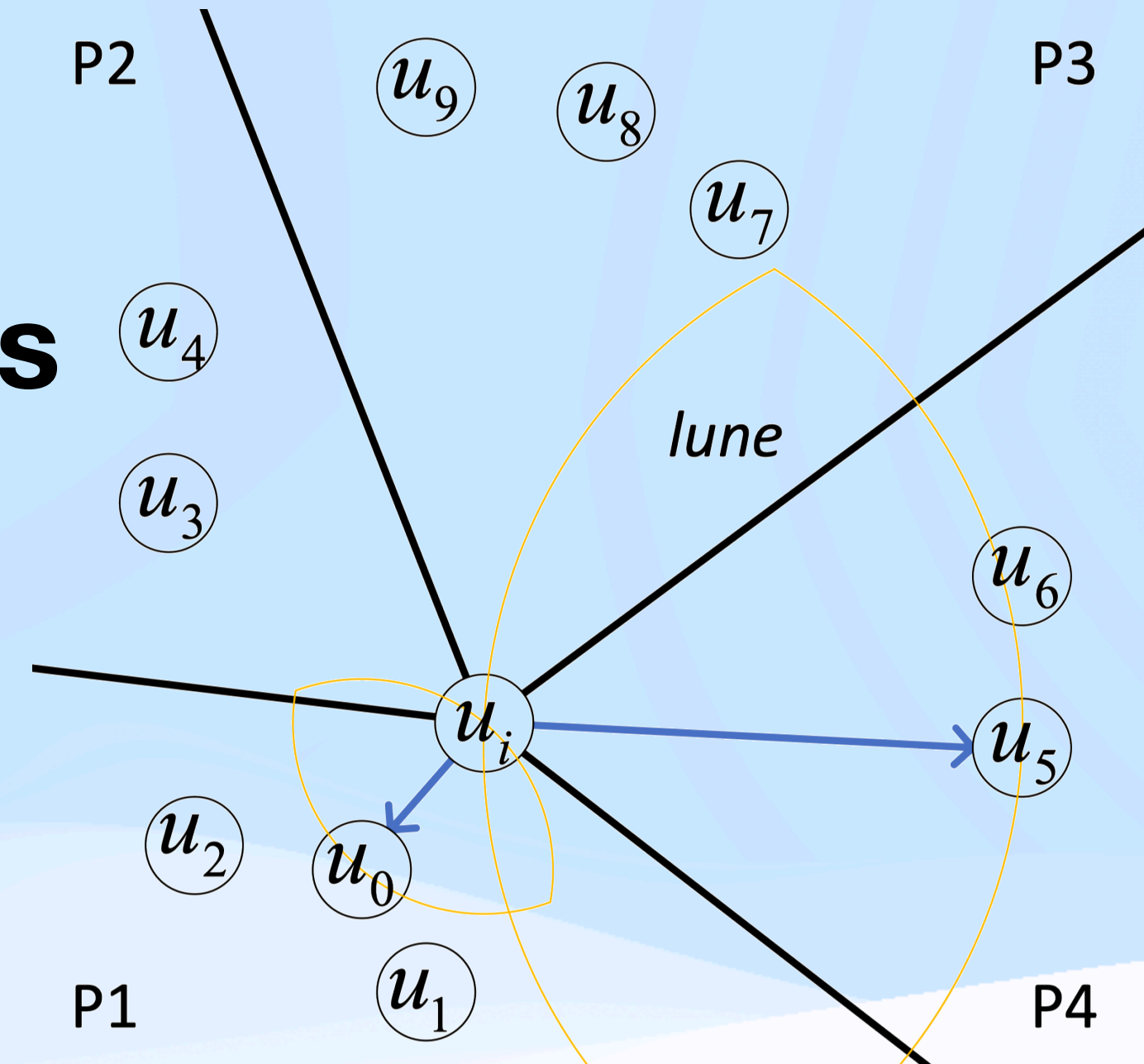
$$\alpha \times \delta_e(x, y) + (1 - \alpha) \times \delta_s(x, y)$$

$$\alpha \times \delta_e(y, z) + (1 - \alpha) \times \delta_s(y, z) < \quad (3)$$

$$\alpha \times \delta_e(x, y) + (1 - \alpha) \times \delta_s(x, y)$$

Examples	$\delta_s(x, y)$	$\delta_e(x, y)$	$\delta_s(x, z)$	$\delta_e(x, z)$	$\delta_s(y, z)$	$\delta_e(y, z)$
(x_1, y_1, z_1)	0.3	0.4	0.8	0.9	0.1	0.7
(x_2, y_2, z_2)	0.5	0.7	0.2	0.4	0.3	0.5
(x_3, y_3, z_3)	0.2	0.6	0.4	0.5	0.3	0.4

Example 1: Table 1 presents several examples to illustrate this challenge. In the first example, (x_1, y_1, z_1) , Equation 2 does not hold for any α value. This means that the edge (x, y) will not be pruned due to the presence of node z according to RNG's pruning strategy. In the second example, (x_2, y_2, z_2) , both equations are satisfied for any $\alpha \in [0, 1]$, indicating that the edge (x_2, y_2) will be consistently pruned by node z_2 , regardless of the α value. In the third example, (x_3, y_3, z_3) , the first equation holds for $\alpha \in [\frac{2}{3}, 1]$ and the second equation holds for $\alpha \in [\frac{1}{3}, 1]$. This means that the edge (x_3, y_3) will be pruned due to the presence of node z when $\alpha \in [\frac{2}{3}, 1]$, as both conditions are satisfied in this range.

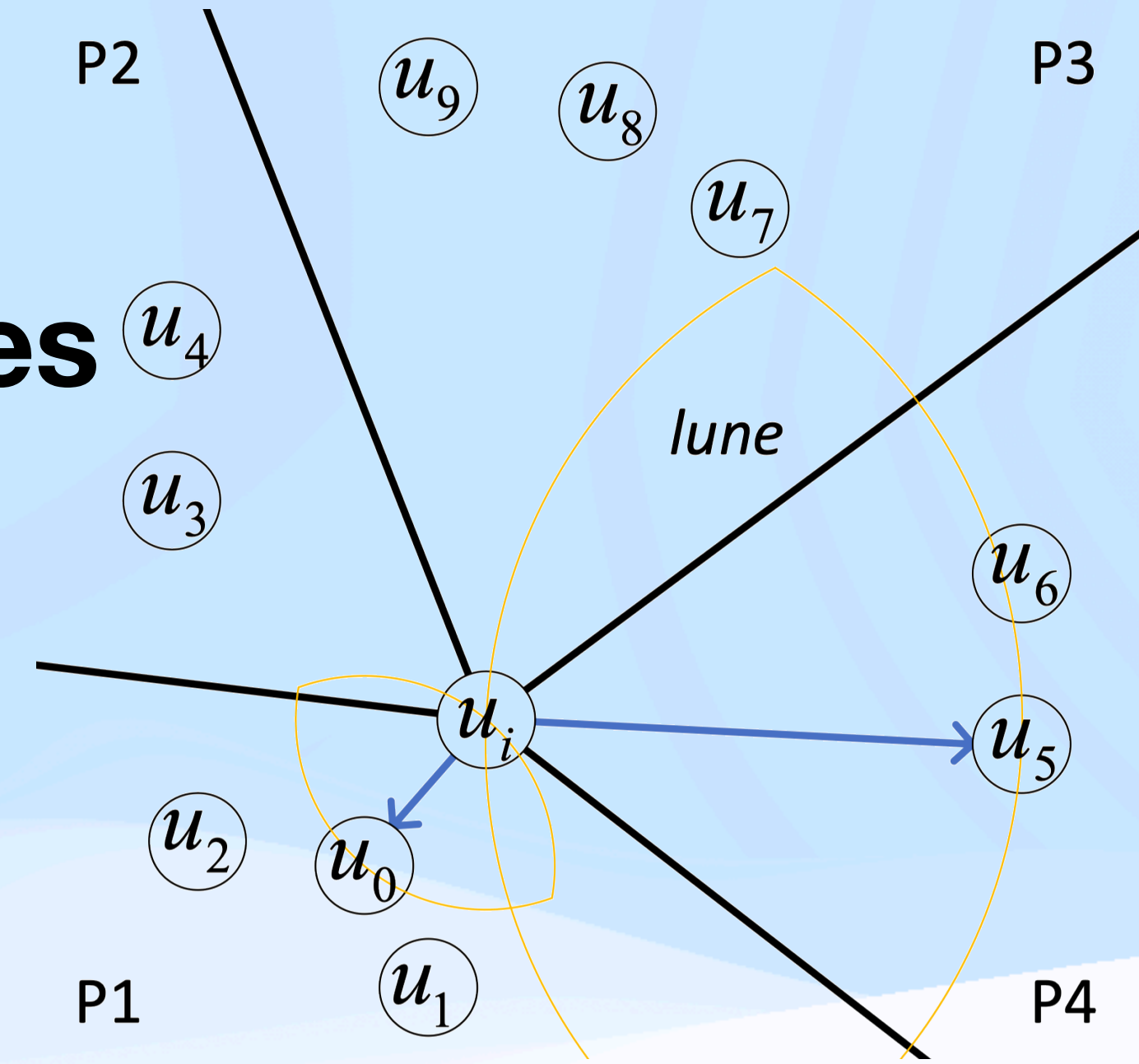


Stage 2. Neighborhood Selection

Dynamic Edge Pruning Strategy: Solve Inequalities

$$\alpha \times (\delta_e(x, z) - \delta_e(x, y) + \delta_s(x, y) - \delta_s(x, z)) < \delta_s(x, y) - \delta_s(x, z) \quad (4)$$

$$\alpha \times (\delta_e(y, z) - \delta_e(x, y) + \delta_s(x, y) - \delta_s(y, z)) < \delta_s(x, y) - \delta_s(y, z) \quad (5)$$



Case 1: If $\delta_s(x, y) - \delta_s(x, z) > 0$ and $\delta_e(x, z) - \delta_e(x, y) + \delta_s(x, y) - \delta_s(x, z) > 0$, then the inequality is satisfied for the range $r_1^z = \left[0, \min \left(1, \frac{\delta_s(x, y) - \delta_s(x, z)}{\delta_e(x, z) - \delta_e(x, y) + \delta_s(x, y) - \delta_s(x, z)}\right)\right]$.

Case 2: If $\delta_s(x, y) - \delta_s(x, z) < 0$ and $\delta_e(x, z) - \delta_e(x, y) + \delta_s(x, y) - \delta_s(x, z) \geq 0$, then the inequality cannot be satisfied for any $\alpha \in [0, 1]$, resulting in $r_1^z = \emptyset$.

Case 3: If $\delta_s(x, y) - \delta_s(x, z) > 0$ and $\delta_e(x, z) - \delta_e(x, y) + \delta_s(x, y) - \delta_s(x, z) \leq 0$, then the inequality can be satisfied for any $\alpha \in [0, 1]$, resulting in $r_1^z = [0, 1]$.

Case 4: If $\delta_s(x, y) - \delta_s(x, z) < 0$ and $\delta_e(x, z) - \delta_e(x, y) + \delta_s(x, y) - \delta_s(x, z) < 0$, then the inequality is satisfied for the range $r_1^z = \left[\min \left(1, \frac{\delta_s(x, y) - \delta_s(x, z)}{\delta_e(x, z) - \delta_e(x, y) + \delta_s(x, y) - \delta_s(x, z)}\right), 1\right]$.

Stage 2. Neighborhood Selection

Dynamic Edge Pruning Strategy

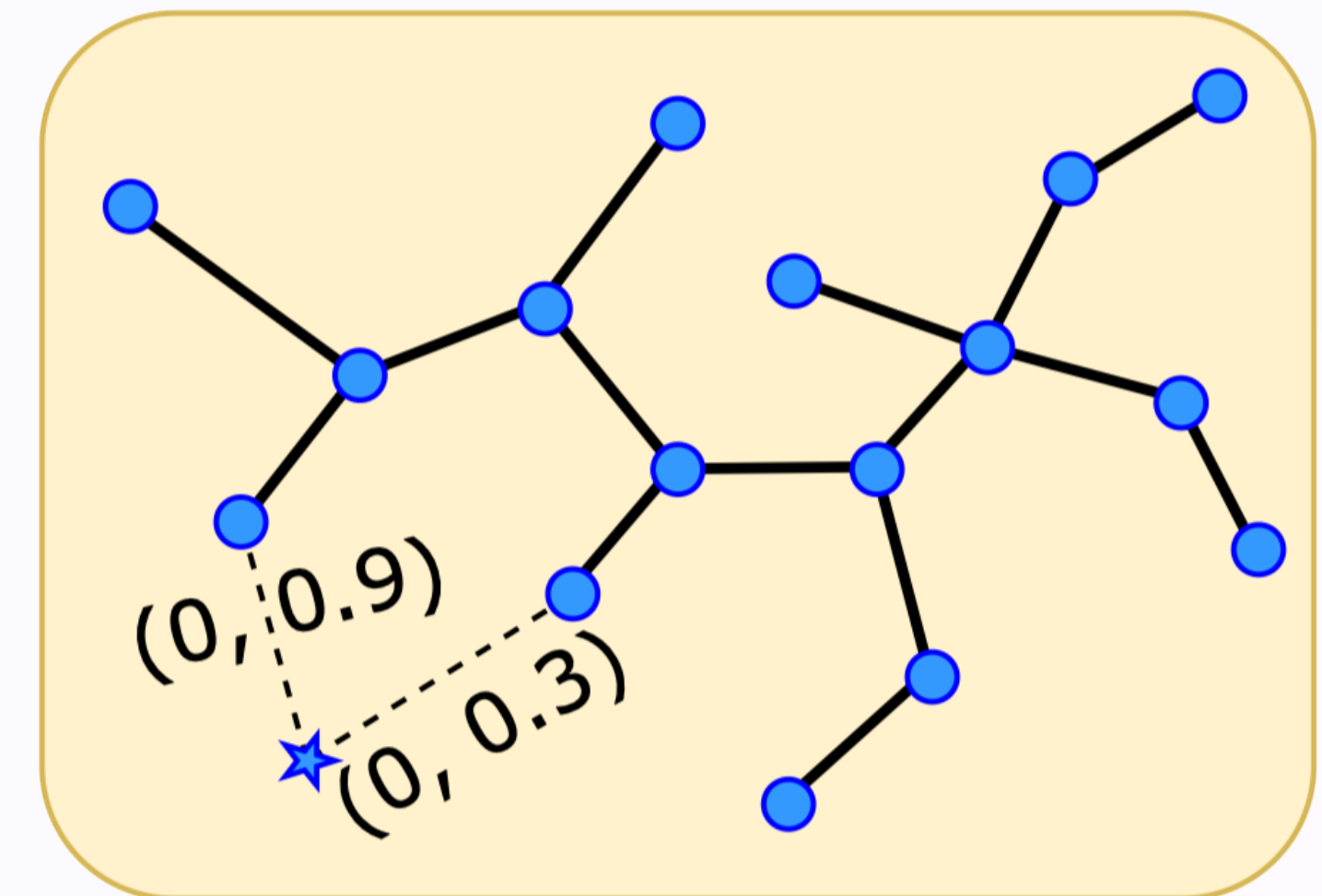
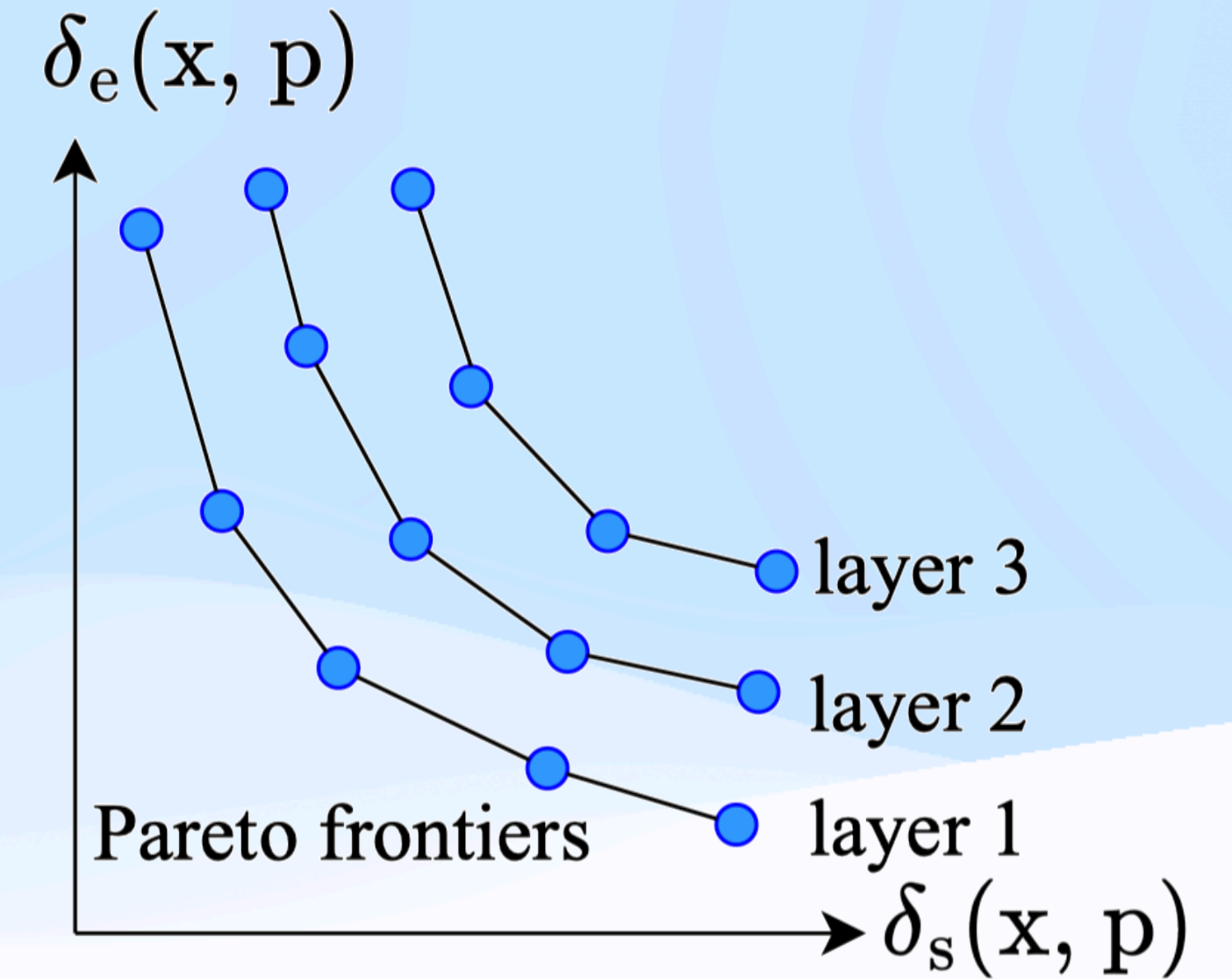
Algorithm 2: Dynamic Edge Pruning Strategy

Input: A candidate set CS consisting of approximate Pareto frontiers, maximum edges M , a threshold value th to prune edges with a small use range

Output: Neighbor Set NS

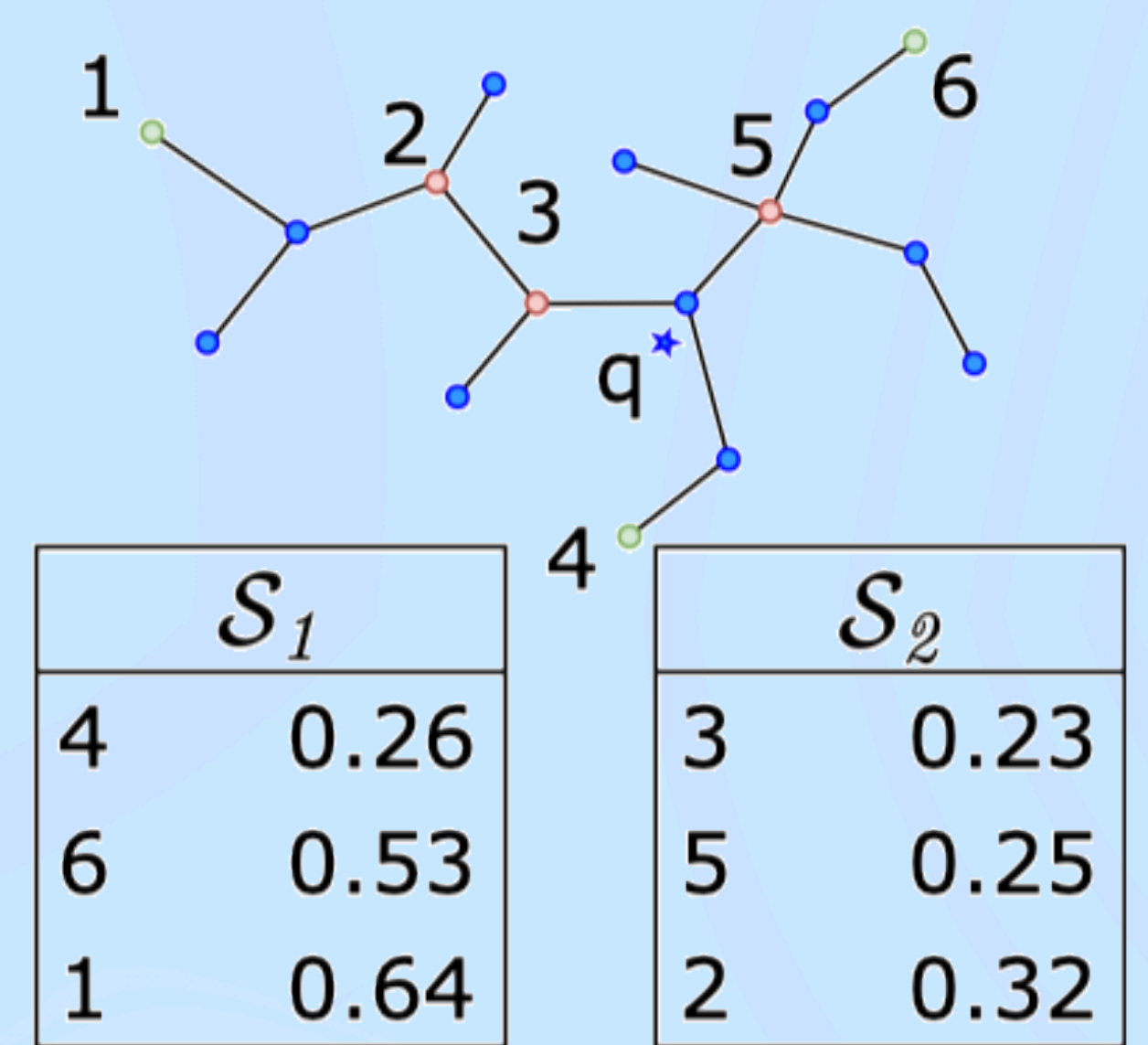
```

1 Procedure DRNGPrune( $CS, M, th$ ):
2   Initialize the Neighbor Set  $NS = \emptyset$ ;
3   foreach  $PF_i \in CS$  do    In Pareto frontiers, iterate from inner to outer layer
4     foreach  $x \in PF_i$  do
5        $r^x \leftarrow \emptyset$ ;
6       foreach  $y \in NS$  do
7         Compute  $r^y$ ;
8          $r[x] \leftarrow r[x] \cup r^y$ ;    Union the pruning range from all triples
9        $u \leftarrow [0, 1] \setminus r[x]$ ;
10      if  $|u| \geq th$  then    Filter range that is too short to be useful in most
11         $NS.add(x, u)$ ;    cases, for instance  $[0, 0.05]$ 
12      if  $|NS| \geq M$  then
13        break;
14  return  $NS$ ;
  
```

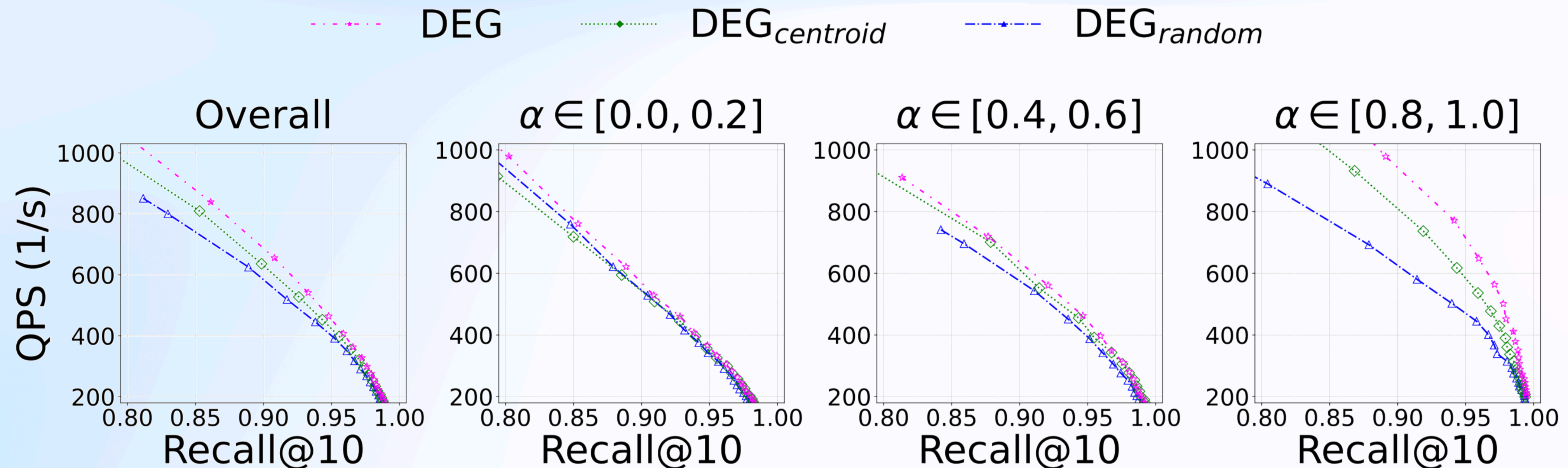


Stage 3. Seed Selection

Current methods: choose seed randomly, or use centroid, which faces problems of **repeating** visiting neighbors.



Experiment shows better to choose edge nodes instead of centroids as seeds.



Summary. Index Construction

Algorithm 3: Index Construction

Input: A dataset D , candidate pool size $ef_{construction}$, maximum edges per node M , a threshold value th to prune edges with a small use range

Output: $G(V, E, ep)$, where ep denotes the seed set

```
1 Procedure DEGBuild( $D, ef_{construction}, M, th$ ):  
2   calculate the centroid  $c$  of  $D$ ;  
3   Initialize graph  $G(V, E, ep)$ ,  $V = \{x_0\}$ ,  $E = \emptyset$ ,  $ep = \{x_0\}$ ;  
4   foreach  $x \in D \setminus \{x_0\}$  do           Build index incrementally, point by point.  
5      $V \leftarrow V \cup \{x\}$ ;  
6      $CS \leftarrow \text{GPS}(G, x, ep, ef_{construction})$ ;       Compute Pareto Frontiers for potential neighbors.  
7      $NS(x) \leftarrow \text{DRNGPrune}(CS, M, th)$ ;           Prune Pareto Frontiers to select neighbors.  
8      $E(x) \leftarrow NS(x)$ ;  
9     foreach  $y \in NS(x)$  do  
10       $E(y) \leftarrow \text{DRNGPrune}(E(y) \cup \{x\}, M, th)$ ;   Prune for nodes that just be connected by  $x$ .  
11      update  $ep$  as the inverse Pareto frontier of  $c$ ;  
12  return  $G$ ;
```

Evaluation

Dataset & Metrics

Text Description — BERT —> vector

Image — ViT —> vector

Video — S3D —> vector

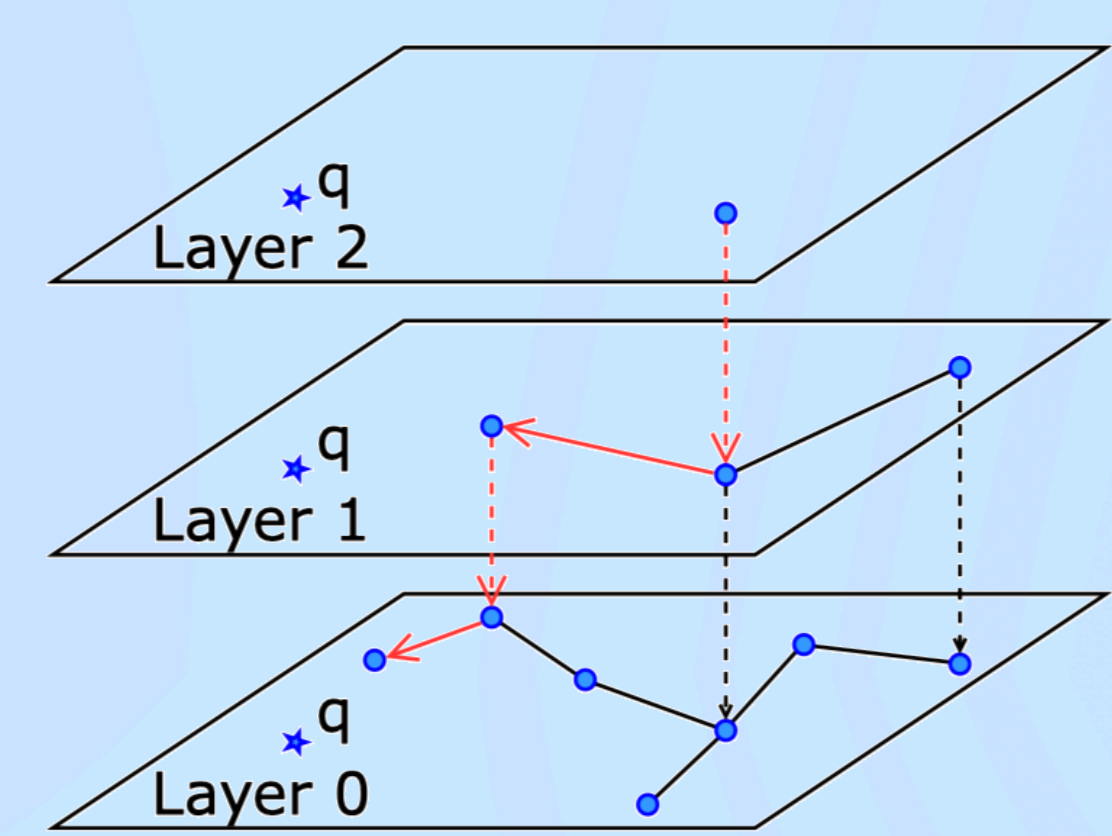
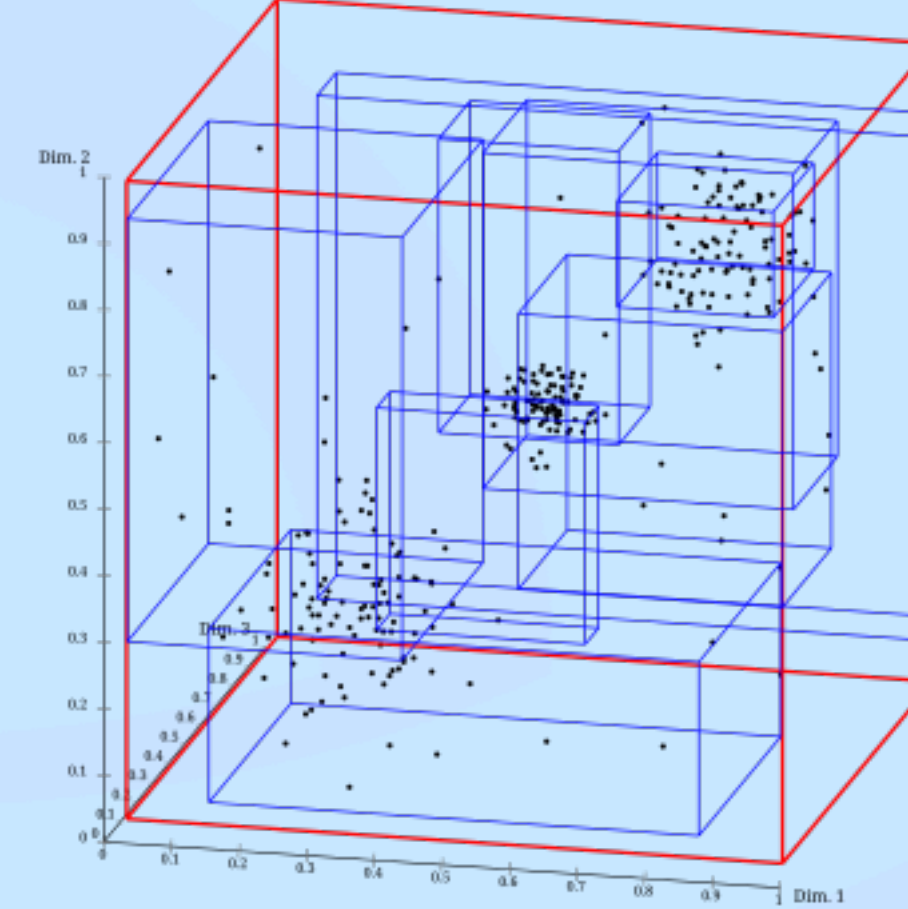
Dataset	$ D $	d	m	$ Q $	Type
OpenImage	507,444	768	768	1,000	Text, Image
Ins-SG	1,000,000	768	2	1,000	Text, Coordinate
Howto100M	1,238,875	1,024	768	1,000	Text, Video
CC3M	3,131,153	768	768	1,000	Text, Image
Twitter-US	10,000,000	768	2	1,000	Text, Coordinate

(OpenImage, CC3M) and (Ins-SG, Twitter-US) will exam scalability of methods.

$$Recall@k = \frac{\mathcal{R} \cap \tilde{\mathcal{R}}}{k}$$

$$QPS = \frac{\#q}{t}$$

Evaluation Baselines

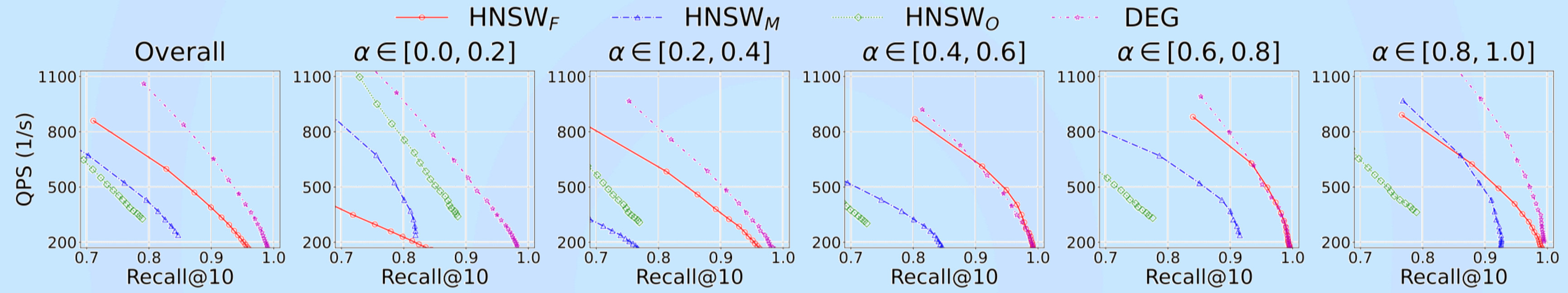


- HNSW-F: HNSF Indexing + Vector Fusion Method (best for $\alpha = 0.5$)
- HNSW-M: HNSF/R-Tree Indexing + Merging Method (Not NRA) (best for $\alpha = 0/1$)

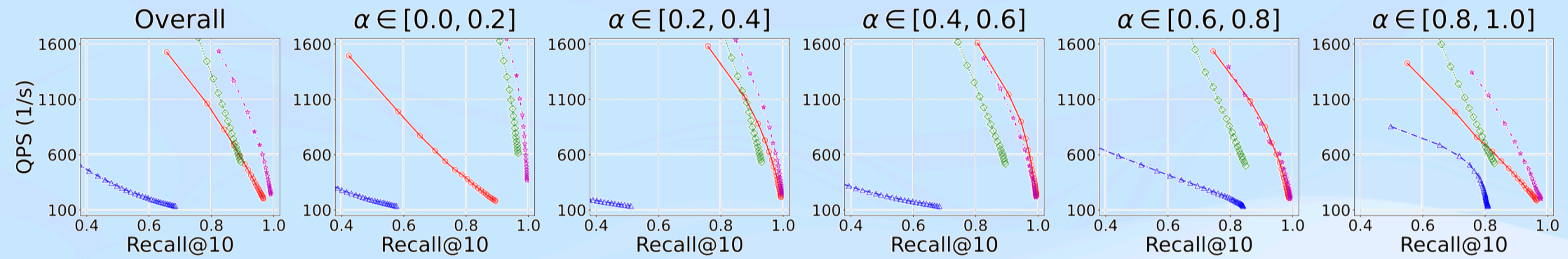
... And Two New Baselines

- HNSW-O: Overlaid indexes from different α values (and restrict the edge number of nodes to be the same levels of others, to avoid unfair comparisons).
- HNSW-Oracle: Use the pre-generated nearest index α .

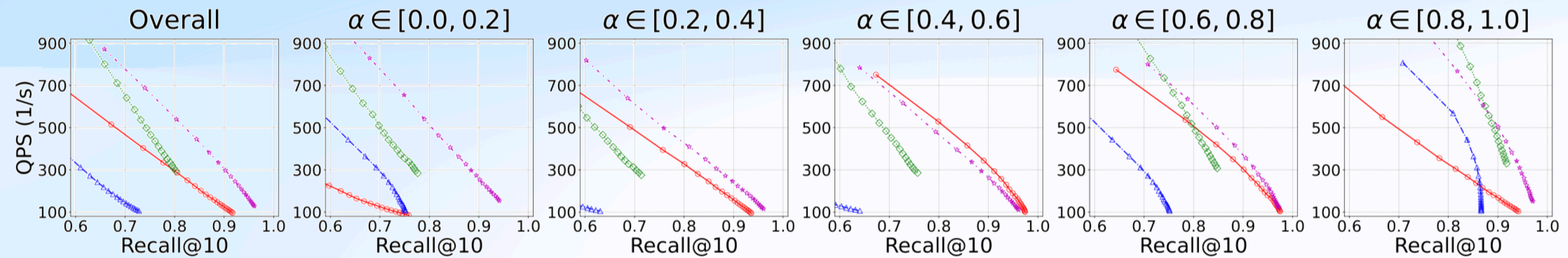
Specifically, α is divided into five intervals in experiment: $[0, 0.2]$, $[0.2, 0.4]$, $[0.4, 0.6]$, $[0.6, 0.8]$ and $[0.8, 1]$, and the pre-generated approaches built indexes on centric of each interval, that is, $\alpha = 0.1, 0.3, 0.5, 0.7, 0.9$.



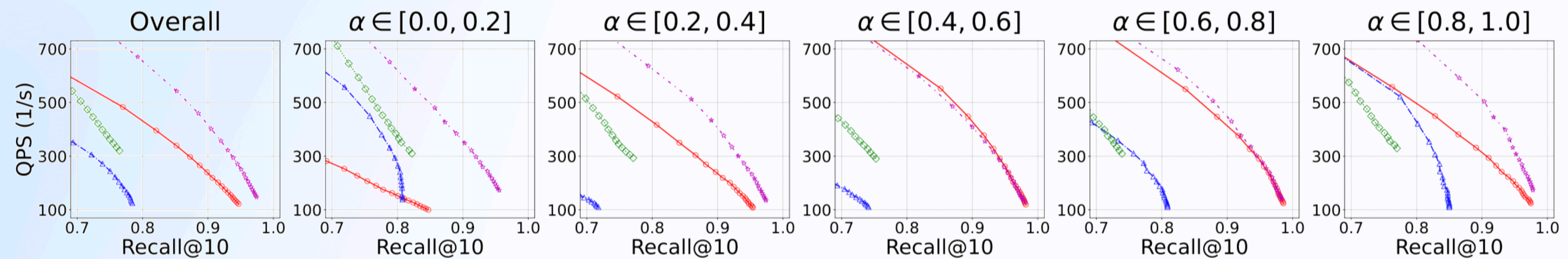
(a) OpenImage



(b) Ins-SG



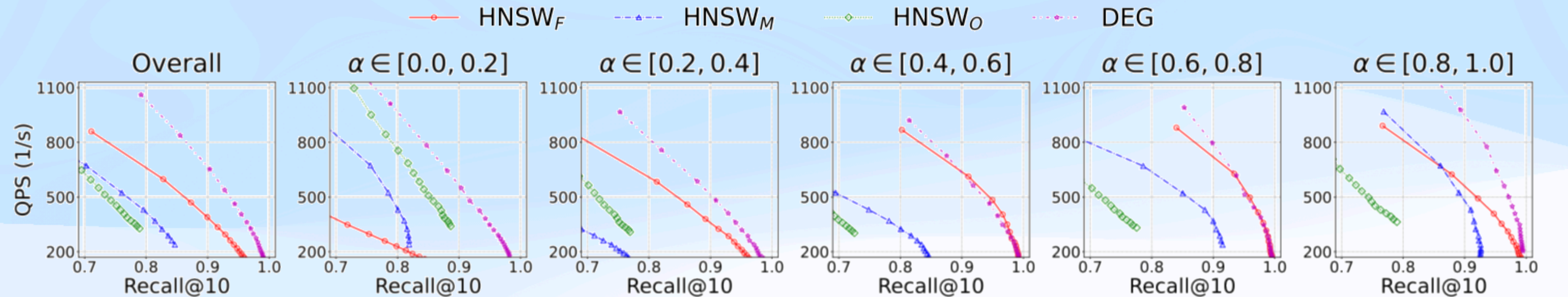
(c) Howto100M



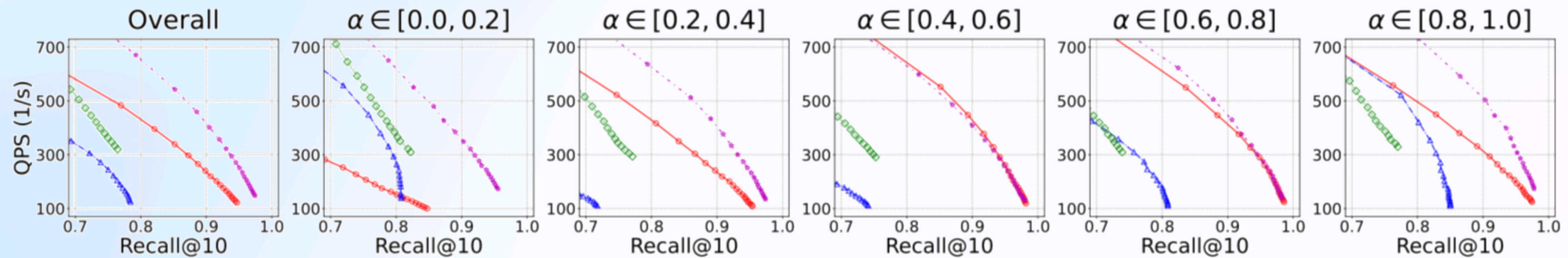
(d) CC3M

1. DEG demonstrates the best performance compared to baselines on all datasets.

Dataset	$ D $	d	m	$ Q $	Type
OpenImage	507,444	768	768	1,000	Text, Image
Ins-SG	1,000,000	768	2	1,000	Text, Coordinate
Howto100M	1,238,875	1,024	768	1,000	Text, Video
CC3M	3,131,153	768	768	1,000	Text, Image
Twitter-US	10,000,000	768	2	1,000	Text, Coordinate



(a) OpenImage

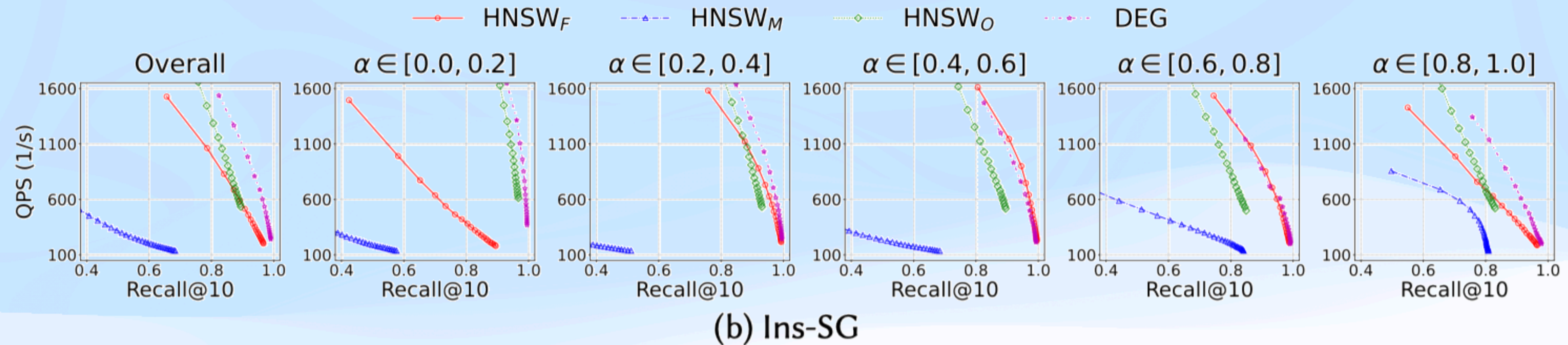


(d) CC3M

2. (Scalability) DEG maintains similar advantages on larger datasets.

Evaluation

Accuracy-Efficiency Trade-off

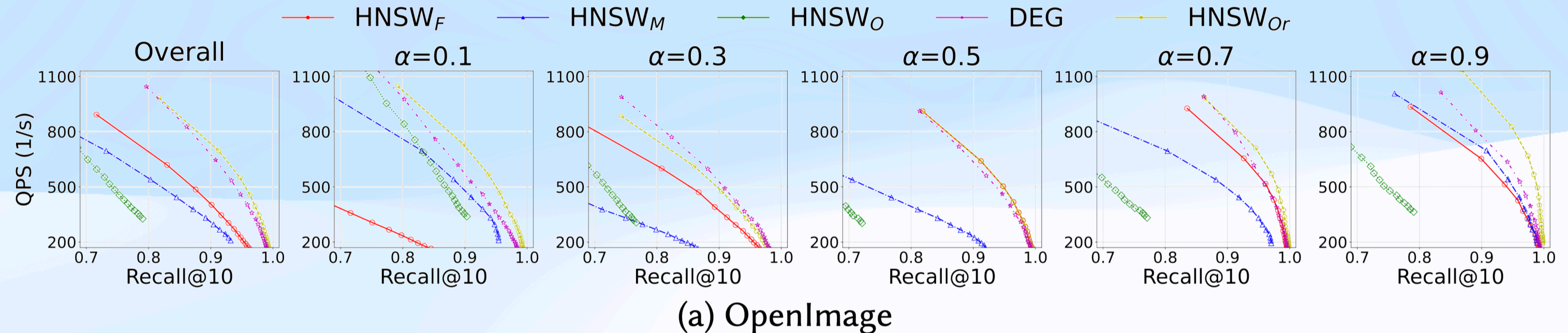


3. HNSW-M performs worse on the Ins-SG dataset, since HNSW-M struggles to retrieve high-quality candidates for reranking.

The inherently **dense distribution of geographic coordinates** compared to high-dimensional vectors, hundreds of objects can coexist within a small spatial scale, making geographically close objects difficult to distinguish from each other.

Evaluation

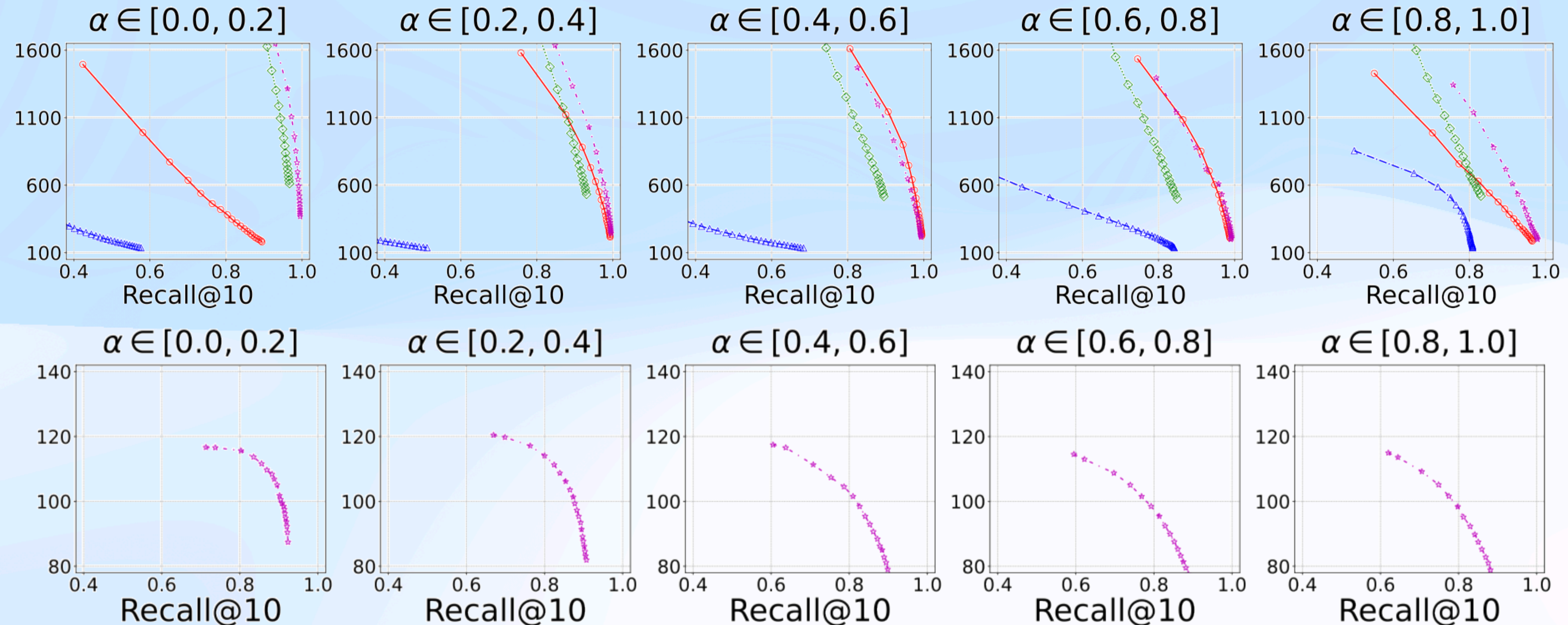
Accuracy-Efficiency Trade-off



4. The overall performance of DEG is comparable to that of HNSW-Oracle and significantly outperforms other baselines.

Evaluation

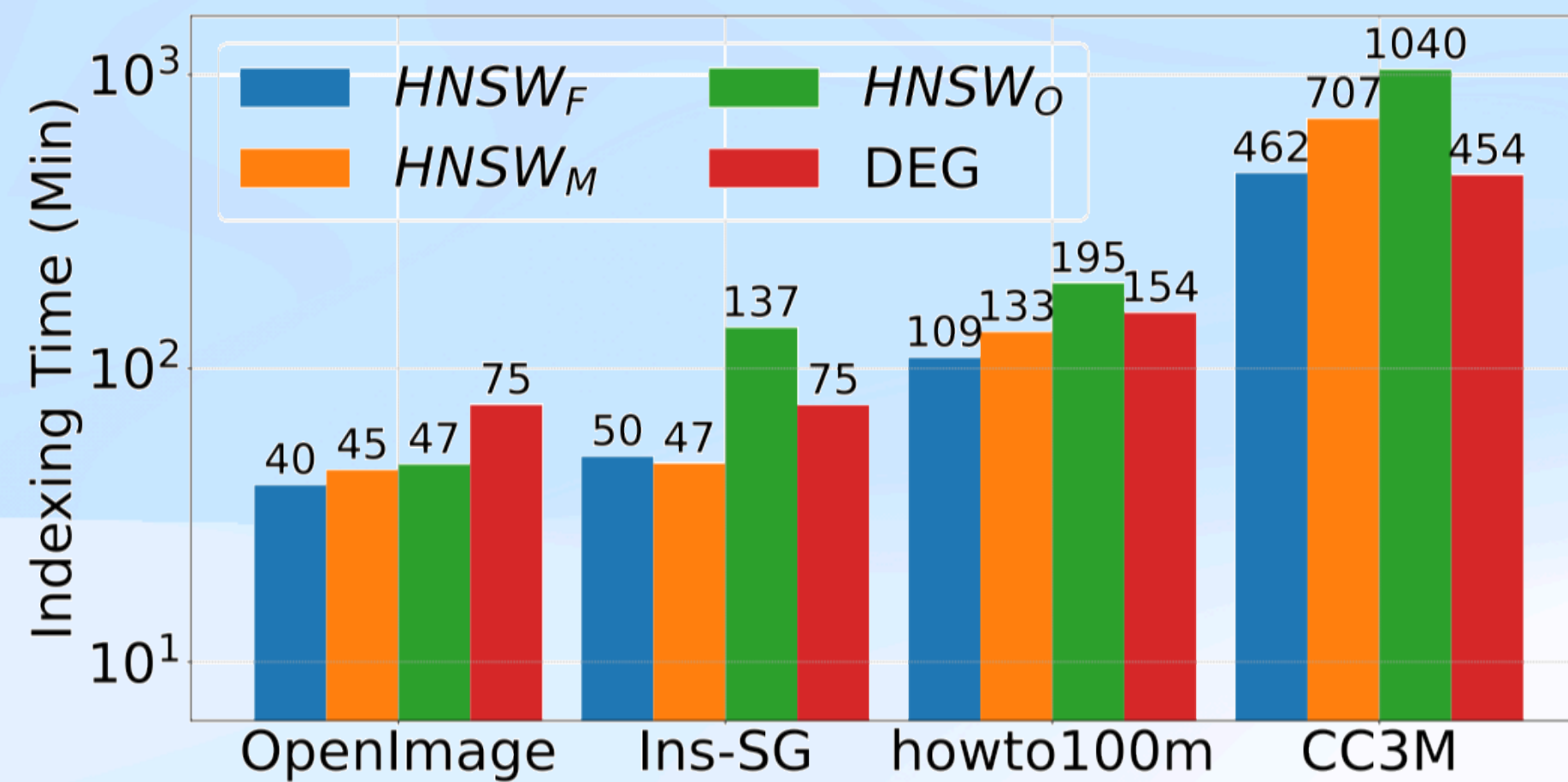
Scalability



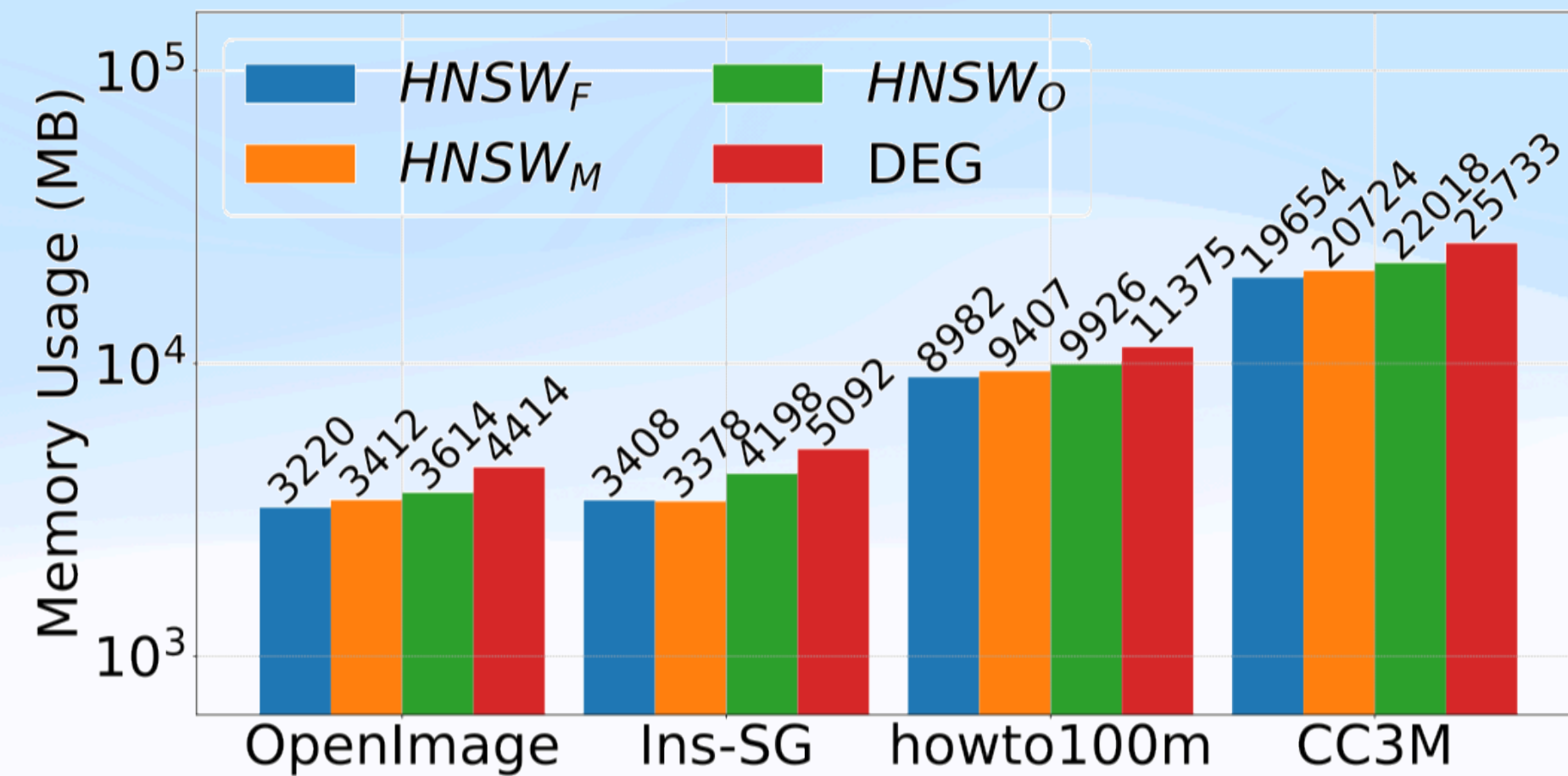
Neither of the baselines completed the index construction within two days, since HNSW's multi-layer mechanism causes its construction cost to increase **exponentially** with the size of the dataset.

Evaluation

Index Construction Cost



(a) Construction Time.



(b) Index Size.

- Comparable Construction Time.
- Slightly Higher Memory Usage.

Evaluation

Ablation Study: Pruning Strategy & Active Range

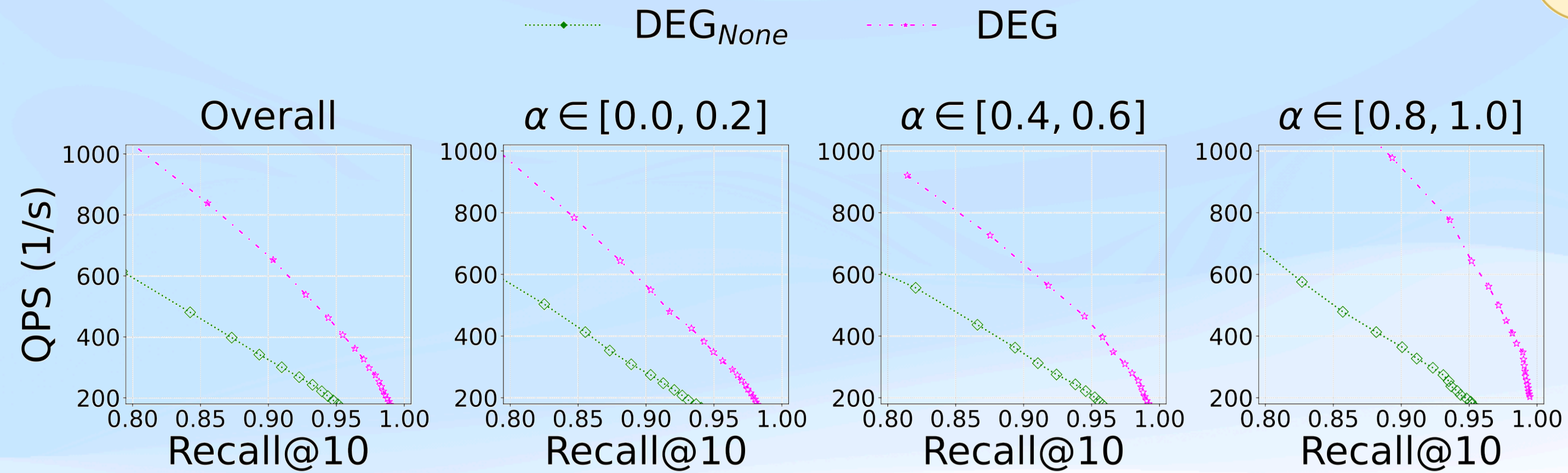
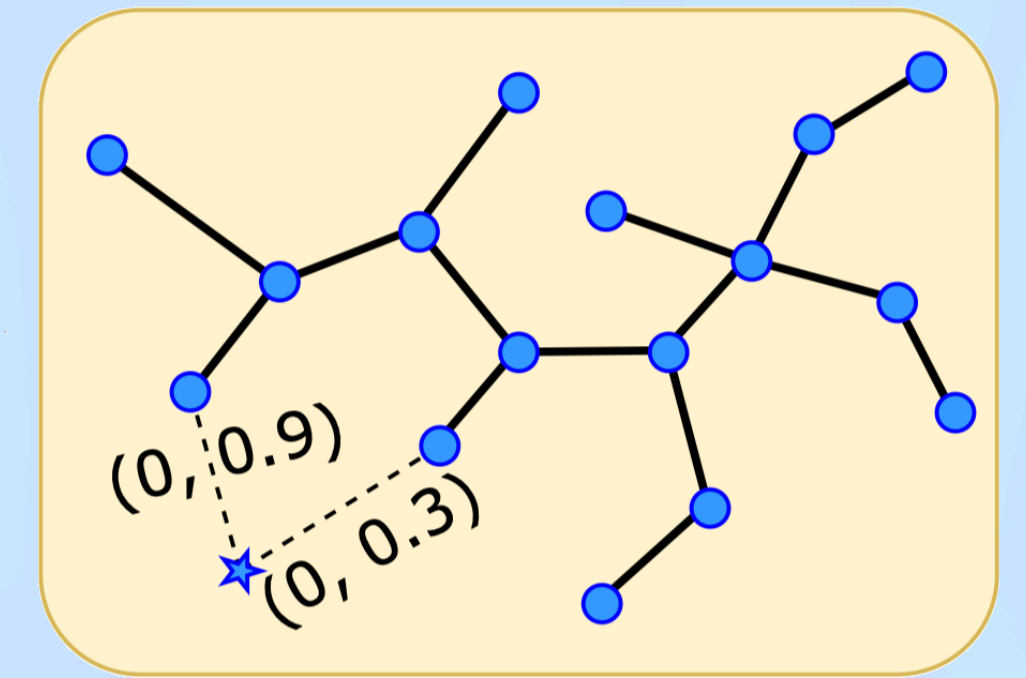


Fig. 10. Ablation Study for the DEG's pruning strategy.

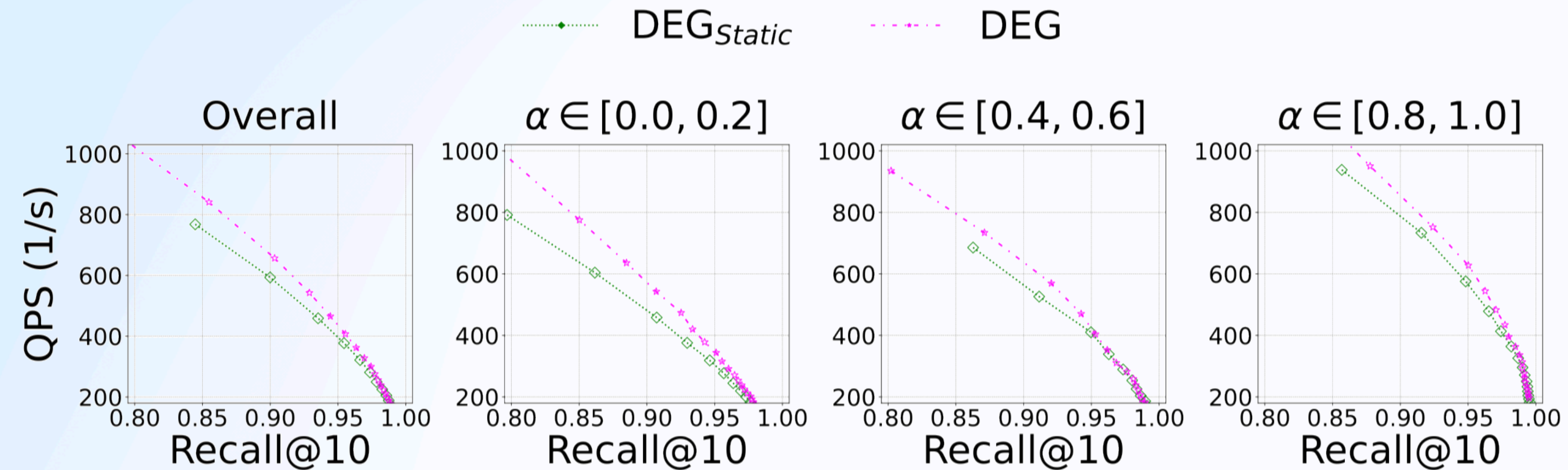


Fig. 11. Ablation Study for the Active Range.

Evaluation

Ablation Study: Candidate Acquisition Method

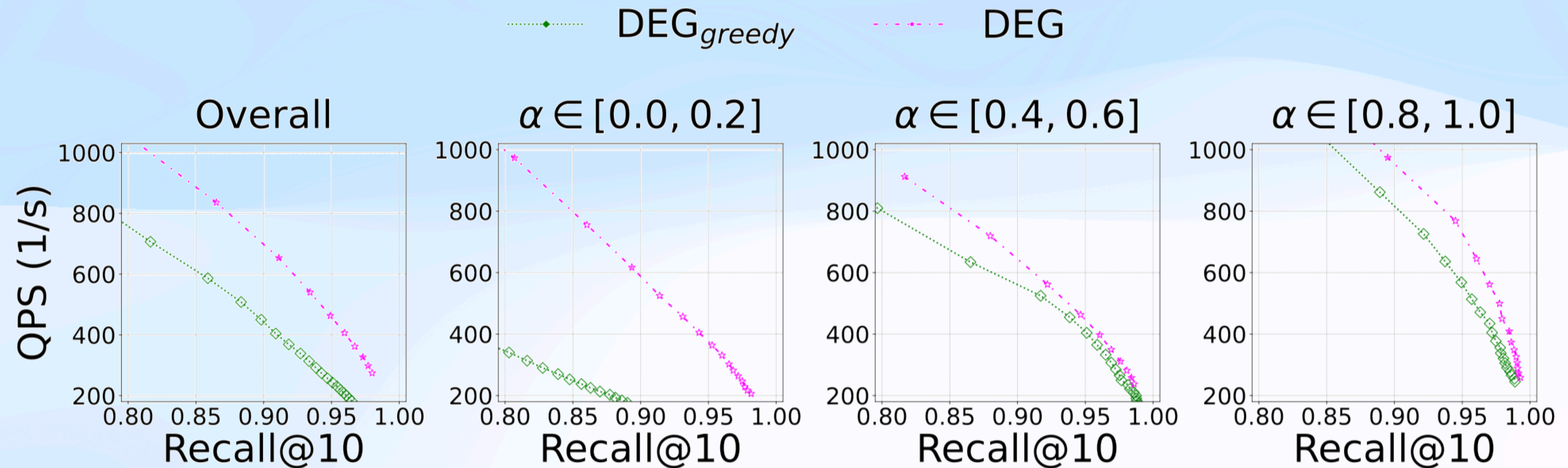
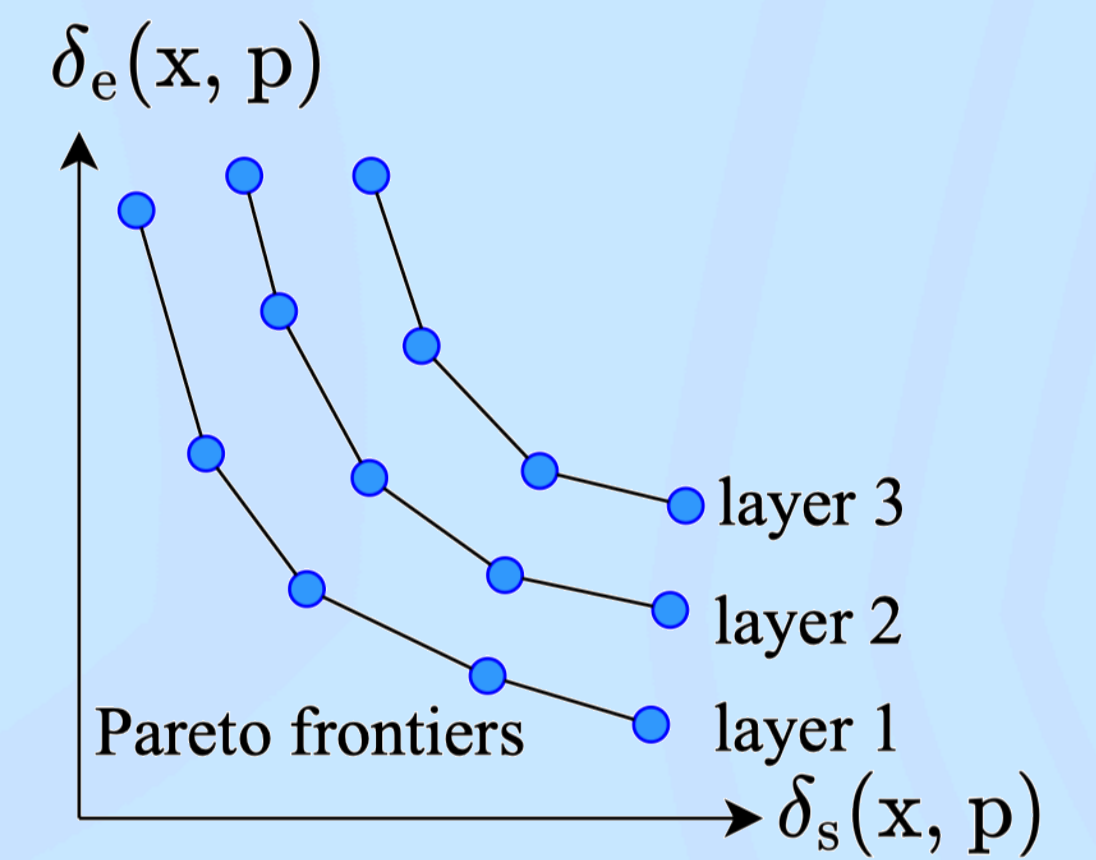
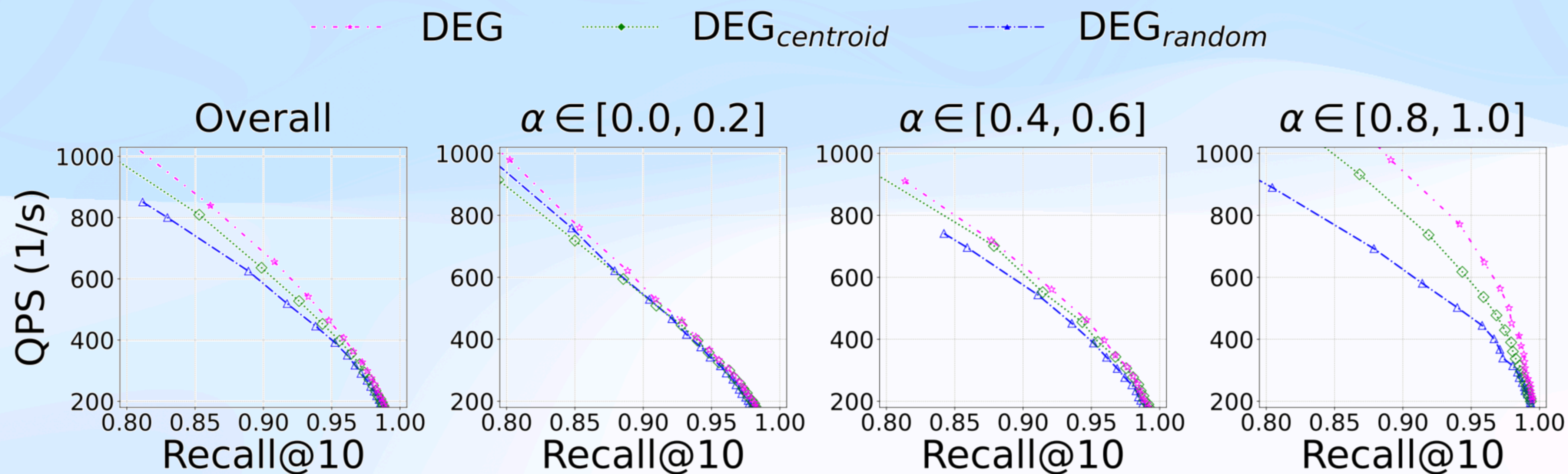
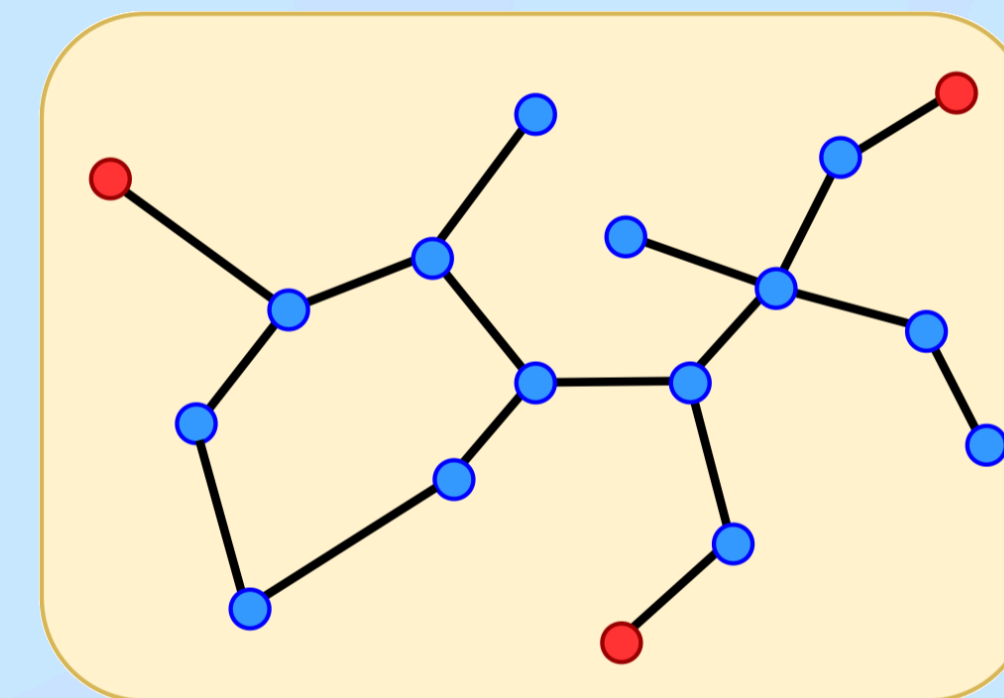


Fig. 12. Ablation Study for candidate acquisition method.

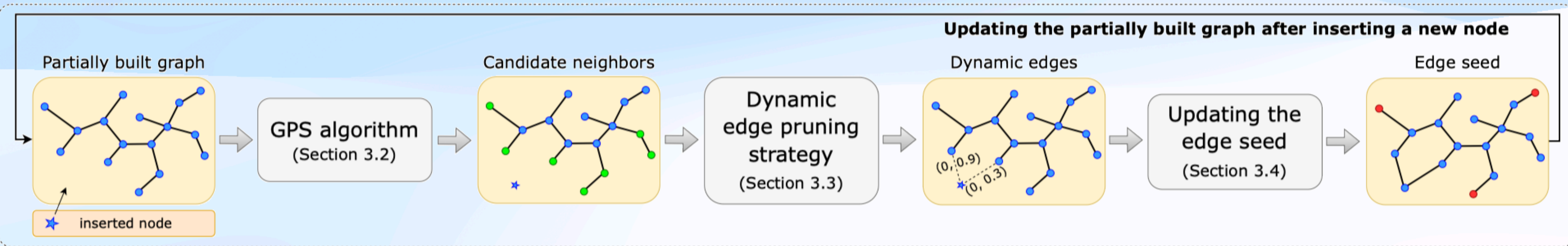
Evaluation

Ablation Study: Edge Seed Acquisition



Future Work

1. Extend index to multi-vector case.
2. Develop a disk-oriented version.



Index Construction: Inserting each node one by one



